

Implementace Bloom filtrů

Bloom Filters Implementation

Richard Bohunovský

Bakalářská práce

Vedoucí práce: doc. Mgr. Jiří Dvorský, Ph.D.

Ostrava, 2025

Zadání bakalářské práce

Student:

Richard Bohunovský

Studijní program:

B0613A140014 Informatika

Téma:

Implementace Bloom filtrů
Bloom Filters Implmentation

Jazyk vypracování:

čeština

Zásady pro vypracování:

Cílem práce je navrhnout, implementovat a experimentálně vyhodnotit knihovnu pravděpodobnostních datových struktur typu Bloom filtr pro platformu .NET a distribuovat ji jako NuGet balíček.

Práce se zaměří na:

- a) správné teoretické nastavení parametrů filtru,
- b) efektivní implementaci (hashování, práce s bity, vláknová bezpečnost dle potřeby),
- c) měření výkonu a přesnosti (propustnost, paměť, míra falešně pozitivních),
- d) srovnání s variantami (Counting, Scalable) a diskusi přínosů.

V bakalářské práci zpracujte tyto části:

1. Nastudujte teoretické základy Bloom filtrů, proveďte rešerši použití v praxi a shrňte motivace/omezení.
2. Zpracujte návrh knihovny.
3. Navrhněte API, konfigurační builder, rozhodněte o reprezentaci bitového pole (vlastní bitová pole vs. třídy třetích stran) a strategii hashování (double hashing apod.).
4. Implementujte efektivní klasický Bloom filtr v C# a ošetřete validaci parametrů, dokumentujte pravděpodobnostní chování.
5. Popište a implementujte volbu parametrů, důsledky na výkon/paměť a proveďte testování a verifikaci.
6. S takto vzniklou implementací proveďte experimenty s cílem testovat výkonnost operací.
7. Zabalte projekt jako NuGet balíček a připravte ukázkovou integraci.

Seznam doporučené odborné literatury:

- [1] MITZENMACHER, M., & UPFAL, E. Probability and Computing: Randomized Algorithms and Probabilistic Analysis. Cambridge University Press, [2005]. ISBN 0521835402
- [2] MUTHUKRISHNAN, S. Data Streams: Algorithms and Applications. Morgan & Claypool, [2005].
- [3] LESKOVEC, J., RAJARAMAN, A., & ULLMAN, J. D. Mining of Massive Datasets (2nd ed.). Cambridge University Press, [2020]. ISBN 978-1-108-47634-8
- [4] CORMEN, Thomas H., Charles Eric LEISERSON, Ronald L. RIVEST a Clifford STEIN. Introduction to algorithms. Fourth edition. Cambridge, Massachusetts: The MIT Press, [2022]. ISBN 978-026-2046-305.

Formální náležitosti a rozsah bakalářské práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí bakalářské práce: **doc. Mgr. Jiří Dvorský, Ph.D.**

Datum zadání: 01.09.2025

Datum odevzdání: 30.04.2026

Garant studijního programu: Ing. Tomáš Fabián, Ph.D.

V IS EDISON zadáno: 14.10.2025 13:39:05

Abstrakt

Tato bakalářská práce zpracovává teoretickou analýzu a praktickou implementaci pravděpodobnostních datových struktur, konkrétně se zaměřuje na Bloomovy filtry a jejich varianty. První část práce je věnována teoretickým východiskům, historickému kontextu a matematickému odvození pravděpodobnosti falešně pozitivních výsledků. Pozornost je zde věnována také optimalizaci paměťových nároků na bitové pole a moderním variantám, jako jsou počítací (Counting) a škálovatelné (Scalable) Bloomovy filtry. Praktická část je následně zaměřena na softwarový návrh a implementaci těchto struktur v jazyce C# ve formě distribuovatelné knihovny (NuGet balíku) optimalizované pro prostředí s podporou více vláken. Výsledná implementace je podrobena testování a výkonnostnímu benchmarkingu. Experimenty vyhodnocují časovou složitost, skutečnou paměťovou efektivitu a reálnou chybovost v porovnání s teoretickými matematickými modely. Práce poskytuje teoretický a praktický pohled na využití Bloomových filtrů.

Klíčová slova

Bloomův filtr; pravděpodobnostní datová struktura; hashovací funkce; softwarová implementace; výkonnostní testování; benchmarking; NuGet balík; falešně pozitivní výsledky

Abstract

This bachelor thesis presents a theoretical analysis and practical implementation of probabilistic data structures, focusing specifically on Bloom filters and their variants. The first part of the thesis is dedicated to the theoretical background, historical context and mathematical derivation of the false positive results probability. Attention is also given to the optimization of memory requirements for the bit array and modern variants, such as Counting and Scalable Bloom filters. The practical part focuses on the software design and implementation of these structures in C# as a distributable library (NuGet package), optimized for thread-safe environment. The resulting implementation is subjected to extensive testing and performance benchmarking. The experiments evaluate the time complexity, actual memory efficiency and real-world error rates in comparison with theoretical mathematical models. Thus, the thesis provides a comprehensive theoretical and practical overview of Bloom filter usage.

Keywords

Bloom filter; probabilistic data structure; hash function; software implementation; performance testing; benchmarking; NuGet package; false positives rate

Poděkování

Rád bych na tomto místě poděkoval vedoucímu práce doc. Mgr. Jiřímu Dvorskému, Ph.D., za odborné vedení a cenné rady, které mi byly v průběhu psaní oporou. Dále bych chtěl poděkovat své rodině za obrovskou podporu, trpělivost a zázemí, které mi během celého studia a psaní této bakalářské práce poskytovala.

Obsah

Seznam použitých symbolů a zkratk	8
Seznam obrázků	9
Seznam tabulek	10
1 Úvod	11
2 Teoretická a technologická východiska	13
2.1 Hardwarový kontext a paměťová hierarchie	13
2.2 Deterministické a pravděpodobnostní datové struktury	14
2.3 Nekryptografické hashovací funkce	14
3 Bloomovy filtry	16
3.1 Historie a kontext vzniku	16
3.2 Anatomie filtru a jeho základní operace	17
3.3 Matematický model pravděpodobnosti falešně pozitivního výsledku	19
3.4 Optimalizace parametrů filtru	20
3.5 Analýza výhod a omezení	21
4 Vývoj a varianty Bloomova filtru	24
4.1 Počítací Bloomův filtr (Counting Bloom Filter)	24
4.2 Škálovatelné Bloomovy filtry (Scalable Bloom Filter)	26
5 Praktická část: Návrh a implementace	27
5.1 Analýza požadavků a metodika vývoje	27
5.2 Architektonický návrh a objektový model	28
5.3 Implementace struktury	31
6 Benchmarking a experimenty	36
6.1 Metodika a prostředí testování	36

6.2	Jednotkové testování	36
6.3	Výkonnostní profilování (Benchmarks)	37
7	Závěr	43
	Literatura	44
	Přílohy	45
A	Velké obrázky a tabulky	46
B	Elektronické přílohy dostupné skrze odevzdávací systém EDISON	51

Seznam použitých zkratek a symbolů

CBF	– Counting Bloom Filter (Počítací Bloomův filtr)
SBF	– Scalable Bloom Filter (Škálovatelný Bloomův filtr)
RAM	– Random Access Memory (Operační paměť)
CPU	– Central Processing Unit (Procesor)
API	– Application Programming Interface (Rozhraní pro programování)
URL	– Uniform Resource Locator (Unikátní textová adresa)
SSD	– Solid State Drive (Pevný disk)
NVMe	– Non-Volatile Memory Express (Expresní Energeticky nezávislá paměť)
AVL	– Adelson-Velsky a Landis (Typ samovyvažujícího se binárního vyhledávacího stromu)
AMD	– Advanced Micro Devices (Americká hardwarová společnost)
NA	– Not Available (V textu označuje hodnotu, která je nedostupná, popř. hodnotu, které nebylo dosaženo.)
A	– Symbol reprezentující průměrný objem paměti v bajtech, který byl během jedné operace dynamicky alokován na spravované haldě.
N, n	– Symbol reprezentující počet prvků vložených do testovací struktury.
ε	– Symbol reprezentující celkovou chybovost (pravděpodobnost falešně pozitivního výsledku) datové struktury.
$\bar{t}$	– Symbol reprezentující průměrný čas potřebný k provedení jedné operace v nanosekundách.
σ_t	– Symbol reprezentující směrodatnou odchylku naměřeného času, udávajícího rozptyl měření a stabilitu výkonu v nanosekundách.

Seznam obrázků

5.1	Use Case diagram znázorňující interakci klientské aplikace s API knihovny	31
6.1	Závislost času operací na kapacitě filtru	38
6.2	Srovnání vlivu velikosti L3 Cache na čas provedení operace Contains	39
6.3	Srovnání vlivu kapacity L3 Cache na čas provedení operace Add	40
6.4	Porovnání rychlosti vkládání u jednotlivých implementací filtru	41
6.5	Rozdíl v časech provedení operace Contains pro pozitivní a negativní výsledek u Škálovatelné verze filtru	42
A.1	Třídní diagram znázorňující modely knihovny	47

Seznam tabulek

6.1	Hardwarové specifikace testovacích stanic	36
A.1	Data výkonostních testů z Pracovního zařízení 1 (Intel Core i7-13700KF)	48
A.2	Data výkonostních testů z Pracovního zařízení 2 (AMD Ryzen 9 5950X)	49
A.3	Data výkonostních testů z Pracovního zařízení 3 (Intel Core i7-1355U)	50

Kapitola 1

Úvod

V dnešní době, která může být nazvána érou Hyper-Scale, se pokoušíme řešit problém testování členství, který je základním kamenem pro práci s daty. Snažíme se tedy zjistit, zda určitá data patří do specifické množiny, či nikoliv. Tento problém byl po dlouhou dobu výpočetní historie řešen převážně deterministicky, s důrazem na matematickou přesnost. To znamená, že tradiční datové struktury byly navrženy tak, aby na otázku, zda se prvek v množině nachází, odpověděly bezchybně a nebylo možné připustit jakoukoliv chybovost.

Nástup fenoménu Big Data, který je charakteristický nejen výrazným zvětšením objemu ukládaných dat, ale také jejich různorodostí a potřebou zvýšit rychlost, jak k těmto datům přistupovat. V důsledku těchto nově vzniklých potřeb bylo poukázáno na obrovské limity deterministických přístupů. V praxi to znamená, že systémy potřebovaly indexovat celý World Wide Web a směřovat terabity síťového provozu v reálném čase. Množství paměti potřebné k udržení stoprocentní přesnosti u těchto miliardových datasetů začalo drasticky překračovat kapacity rychlých operačních pamětí, jako jsou například paměti RAM. Tento vývoj postavil programátory a softwarové inženýry před dilema: buďto udržet absolutní přesnost za cenu celkového zpomalení systému vlivem diskových operací, místo operací prováděných pamětmi RAM, nebo přehodnotit samotné požadavky na testování členství.

Kompromisem se staly pravděpodobnostní datové struktury, jako je například Bloomův filtr [1]. Tyto struktury jsou schopny obětovat absolutní přesnost a připouštějí kontrolovatelné množství falešně pozitivních chyb, výměnou za ohromnou úsporu paměti a konstantní časovou složitost. Bloomovy filtry jsou jedny z nejvíce využívaných pravděpodobnostních datových struktur na testování členství, a proto se jimi budeme dále podrobně zabývat.

Hlavním cílem této bakalářské práce je kompletní analýza Bloomových filtrů a jejich následná implementace do podoby optimalizované, distribuovatelné knihovny. Výsledkem praktické části je knihovna programovaná v jazyce C#, ve formě NuGet balíku. Knihovna je navržena s ohledem na maximální výkon, využitím rychlých nekryptografických hashovacích funkcí a plně optimalizována pro nasazení v prostředí s více vlákny (thread-safe).

Práce je rozdělena do logických celků, které na sebe navazují. Kapitola 2 definuje teoretická východiska, hardwarový kontext doby a matematický model nezbytný pro pochopení algoritmů, které Bloomovy filtry používají. Dále se v kapitole 3 zaměříme specificky na strukturu Bloomových filtrů. Jejich moderní vylepšené varianty, jako jsou Počítací Bloomovy filtry (CBF) [2], nebo Škálovatelné Bloomovy filtry (SBF) [3] jsou podrobně rozebrány v kapitole 4. Následující kapitoly se detailně věnují praktické části mé bakalářské práce, jako je softwarový návrh výsledné knihovny a samotná implementace datové struktury. Závěrečná část práce je vyčleněna experimentům a benchmarkingu, kde je knihovna podrobena zátěžovým testům pro ověření její časové a paměťové složitosti.

Kapitola 2

Teoretická a technologická východiska

Pro pochopení motivace vzniku pravděpodobnostních struktur, mezi které zapadají i Bloomovy filtry, je nutné nejprve objasnit limity, které mají deterministické přístupy, a proč je žádoucí tyto alternativy využívat. Dále je potřeba definovat vlastnosti hashovacích funkcí, které tvoří jejich výpočetní základ.

2.1 Hardwarový kontext a paměťová hierarchie

Moderní výpočetní architektury se neobejdou bez propracované paměťové hierarchie, vytvořené optimálním kompromisem mezi kapacitou, náklady a přístupovou dobou. Přestože výkon procesorů neustále roste, rychlost pamětí tomuto tempu historicky nestačí. Rostoucí rozdíl mezi hrubým výpočetním výkonem a dobou odezvy paměti tak představuje kritické úzké hrdlo celého systému [4].

Komunikace mezi operační pamětí (RAM) a sekundárním úložištěm (např. SSD nebo NVMe) tvoří u komplikovaných datových struktur podstatnou výkonnostní bariéru. Důvodem je nesrovnatelný rozdíl v rychlosti čtení. Procesor zvládne přistoupit do RAM během desítek nanosekund, u disků se však doba čekání prodlužuje na mikrosekundy až milisekundy. Jedná se tedy o nárůst latence v řádech několika jednotek. Tento problém lze jasně pozorovat v momentě, kdy velikost indexovatelných dat přesáhne dostupné místo v RAM – typicky to jsou registry škodlivých URL adres, které obsahují stovky milionů až miliardy záznamů [5]. Nutnost odkládání dat na sekundární úložiště má za následek obrovskou degradaci propustnosti vstupně-výstupních operací. Pro systémy, které vyžadují minimální latenci, jako jsou prvky síťové nebo analytické databáze v reálném čase, je takový stav velmi nežádoucí. Aby tyto systémy mohly plnit svůj účel, musí jejich datové struktury trvale přetrvávat v paměti RAM. V nejlepším případě jsou uloženy přímo uvnitř cache procesoru.

2.2 Deterministické a pravděpodobnostní datové struktury

Hlavním cílem testování členství je schopnost se algoritmicky rozhodnout, zda konkrétní prvek x patří do definované množiny R . Klasické řešení tohoto problému zahrnovalo deterministické přístupy, jejichž základním rysem je absolutní matematická přesnost a nepřipouští žádnou chybovost.

2.2.1 Deterministické struktury

Datové struktury, jako hashovací tabulky a binární stromy (včetně jejich vyvážených variant, jako jsou Red-Black stromy [6, str. 432–487] nebo AVL [6, str. 452–454]), poskytují při testování členství zcela jednoznačnou odpověď, zda se prvek v množině nachází, či nikoliv. Tato jistota správného výsledku ovšem vyžaduje fyzikální uložení buď samotných dat (například kompletních URL adres), nebo příslušných ukazatelů. Následkem toho roste paměťová složitost těchto datových struktur lineárně vzhledem k počtu uložených prvků n – jedná se tedy o složitost $\mathcal{O}(n)$. U masivních objemů dat se tak dostupná RAM zaplní poměrně rychle. Operační systém poté musí odkládat informace na vedlejší pomalé diskové úložiště. Tím se vracíme k problému drastického zpomalení, které bylo detailněji popsáno v úvodu.

2.2.2 Pravděpodobnostní struktury

Zcela odlišný rámec nabízí nedeterministické, neboli pravděpodobnostní datové struktury. Jejich základním principem je, že do paměti neukládají úplná původní data. Místo nich se v paměti uchovávají pouze jejich značně komprimované matematické reprezentace. Přejít na tento model pochopitelně vyžaduje jistý ústupek. Extrémní úspora kapacity, kdy paměťové nároky klesají i o několik řádů, je vyměněna za toleranci vůči specifické chybovosti. Z hlediska logické stránky testování členství tyto struktury připouštějí vznik falešně pozitivních výsledků (tj. prvek je označen za přítomný, ačkoliv se v množině reálně nevyskytuje). Výskyt falešně negativních výsledků je naproti tomu matematicky vyloučen. Kromě algoritmů jako *HyperLogLog* [7] pro odhad kardinality či *Count-Min Sketch* [8] pro frekvenční analýzu, spadají do této kategorie především Bloomovy filtry pro testování členství.

2.3 Nekryptografické hashovací funkce

Hashovací operace představují výpočetní jádro všech nedeterministických datových struktur. Transformace dat u těchto funkcí stojí na jednoznačném principu: algoritmus vezme vstup o libovolné velikosti a převede ho na výstupní řetězec pevně dané délky. Tento výsledek hashování se běžně nazývá výstupním hashem či otiskem. Pro dosažení maximální efektivity u Bloomových filtrů je naprosto klíčové zvolit správnou rodinu hashovacích algoritmů.

Z pohledu softwarového inženýrství se tyto funkce mohou dělit do dvou primárních kategorií.

- **Kryptografické hashovací funkce (např. SHA-256 [9]):** Jejich hlavním cílem je bezpečnost hashovaných dat. Jsou navrženy s důrazem na jednosměrnost, jakmile jsou jednou hashována, nejde z nich získat původní vstup. Tato přidaná hodnota je ovšem vyměněna za vysokou výpočetní režii a značnou zátěž procesoru.
- **Nekryptografické hashovací funkce (např. MurmurHash [10], xxHash [11]):** Tyto algoritmy naopak žádnou bezpečnostní záruku proti potenciálním hrozbám a útokům neposkytují. Avšak při operacích, které probíhají výhradně v RAM nebo cache procesoru, tato skutečnost nevytváří žádné riziko. Jejich jediným účelem je maximalizace výpočetní propustnosti při zachování lavinového efektu (tj. i změna o jediný bit způsobí zcela odlišný a nepředvídatelný výstupní hash). Tato vlastnost je v kombinaci s rovnoměrným rozložením dat efektivní v ochraně před shlukováním dat a minimalizací rizika kolizí.

Na samotnou implementaci Bloomových filtrů jsou kryptografické funkce prakticky nepoužitelné. Jejich výpočetní náročnost by degradovala jinak rychlé vyhledávání s konstantní časovou složitostí $\mathcal{O}(k)$, kde k představuje počet využitých hashovacích funkcí. Moderní systémy i praktická část této bakalářské práce se proto spoléhají na rychlé nekryptografické alternativy. Ty jsou velmi často optimalizované na specifickou procesorovou architekturu, díky čemuž zajišťují rovnoměrné rozprostření otisků v bitovém poli s minimální možnou latencí.

Kapitola 3

Bloomovy filtry

V této kapitole se budeme detailněji věnovat předmětu této bakalářské práce – datové struktuře Bloomova filtru. Kapitola začíná historickým kontextem a hardwarovou motivací, která vedla k jejímu vzniku. Následuje vnitřní anatomie filtru, jeho elementární operace a princip, na kterém garantuje absenci negativních výsledků.

3.1 Historie a kontext vzniku

Bloomovy filtry vznikly jako přímá odpověď na drastická hardwarová omezení konce šedesátých let. Tento dobový problém detailněji rozebral Burton Howard Bloom ve svém průlomovém článku *Space/time trade-offs in hash coding with allowable errors* [1], který vyšel roku 1970. Tehdejší sálové počítače trpěly extrémním nedostatkem operační paměti. Její mimořádně vysoká cena navíc značně omezovala velikost jakýchkoliv datových struktur udržovaných přímo uvnitř. Sekundární úložiště, tehdy převážně magnetické disky a bubnové paměti, sice poskytovala mnohem větší kapacitu, ale jejich mechanická podstata znamenala znatelně pomalejší přístupovou dobu.

Bloom se ve svém výzkumu zabýval automatizovaným zpracováním textu. Konkrétně řešil algoritmy pro dělení slov na konci řádků, přičemž cílem jeho výzkumu bylo dosáhnout bezchybného, ale i okamžitého výsledku. Když zkoumal analýzu anglického jazyka, všiml si zajímavé statistické asymetrie. Zhruba 90 % všech slov totiž podléhá jednoduchým a jasně daným pravidlům, takže je lze bez problémů rozdělit algoritmicky. Zbýlých 10 % však tvoří výjimky. K jejich správnému rozdělení je naprosto nezbytné nahlédnout do specializovaného slovníku.

Vytvoření kompletního referenčního slovníku všech slov (šlo o zhruba 500 000) ovšem vygenerovalo datovou sadu, která dostupnou kapacitu operační paměti v tehdejší době překračovala. Situaci nešlo vyřešit ani neustálým dotazováním na pomalý mechanický disk. Pokud by totiž systém musel u každého jednotlivého slova zjišťovat, zda nepatří mezi výjimky, ztratil by tím zcela neakceptovatelné množství výpočetního času.

Možným východiskem, které Bloom navrhl, spočívalo v nahrazení bezchybné deterministické hashovací tabulky v operační paměti jiným, vysoce komprimovaným pravděpodobnostním modelem. Samotný autor tento koncept původně označil jako „Metodu 2“. Tato nová struktura dokázala reprezentovat celou databázi nepravidelných slov přímo v RAM, a to díky jedinému zásadnímu ústupku: systém záměrně připustil kontrolovanou míru chybovosti výměnou za úsporu úložného prostoru.

Vnitřní logika této původní struktury stála při testování jakéhokoliv slova na třech základních scénářích:

- **Skutečně negativní výsledek (True Negative):** Pokud filtr slovo odmítne (vyhodnotí, že do množiny výjimek nepatří), systém na něj aplikuje standardní algoritmická pravidla pro dělení slov. Jelikož filtr garantuje absolutní absenci falešně negativních chyb, nikdy se nestane, že by nepravidelné slovo prošlo běžnými pravidly.
- **Skutečně pozitivní výsledek (True Positive):** Jestliže filtr slovo přijme (vyhodnotí jej jako výjimku), systém sáhne na pevný disk a ze slovníku načte jeho správný vzor dělení.
- **Falešně pozitivní výsledek (False Positive):** V případě, že filtr chybně označí zcela běžné slovo za výjimku, systém sice provede dotaz na diskové úložiště, ale vzápětí zjistí, že slovo ve slovníku chybí. Jakmile tuto skutečnost zjistí, jednoduše se vrátí zpět na standardní algoritmická pravidla. Tento proces znamená, že systém provede jedno zbytečné čtení z disku navíc. Jde ale o naprosto přijatelnou daň za to, že systém nevygeneruje žádnou falešně negativní chybu.

Tento výzkum v praxi dokázal ohromný potenciál obětování malého, snadno kontrolovatelného množství přesnosti za účelem eliminace drahých přístupů na sekundární úložiště. Ve světě výpočetních technologií tak definoval zcela nový a fundamentální princip: možnost strategického vyvažování mezi rychlostí samotného vykonávání programu, velikostí jeho paměťové reprezentace a kontrolovanou pravděpodobností falešně pozitivních výsledků.

3.2 Anatomie filtru a jeho základní operace

Bloomův filtr funguje na principu aplikace hashovacích funkcí nad bitovými vektory. Na rozdíl od deterministických datových struktur do sebe nikdy neukládá samotná zdrojová data. Jejich přítomnost v datové sadě je reprezentována pomocí otisků, vypočítaných matematicky pomocí oněch hashovacích funkcí.

Z hlediska vnitřního uspořádání se standardní Bloomův filtr skládá ze dvou základních komponent:

1. **Bitové pole:** Pole o velikosti m bitů, standardně označované jako B , ve kterém jsou všechny bity na počátku inicializovány na hodnotu 0.

2. **Hashovací funkce:** Předem definovaná sada k nezávislých hashovacích funkcí, označovaných jako $h_1, h_2, \dots, h_k$, nebo jako množina nezávislých hashovacích funkcí H .

Jak ve svých výzkumech upozorňují Mitzenmacher a Upfal [12], je naprosto nezbytným matematickým předpokladem, aby k hashovacích funkcí Bloomova filtru rozprostírala prvky napříč bitovým polem maximálně rovnoměrně a nezávisle na sobě. Jen tak lze minimalizovat shlukování dat, které je pro Bloomův filtr nežádoucí. Případné shlukování by totiž znatelně zvýšilo pravděpodobnost falešně pozitivních výsledků, jelikož by se otisky různých prvků začaly překrývat. Každá funkce proto musí vstupnímu prvku přiřadit jeden konkrétní index z intervalu $\{0, \dots, m-1\}$.

3.2.1 Operace vkládání

Samotné plnění filtru probíhá čistě akumulativně. Pokud chceme do množiny vložit nový prvek x , systém ho nejprve musí předat každé z k nezávislých hashovacích funkcí, čímž získáme k různých hashovacích hodnot. Na každou z těchto hodnot poté aplikujeme operaci modulo (dělení velikostí bitového pole m). Zbytek po tomto dělení nám určuje přesný index uvnitř bitového pole. Bity na těchto pozicích se nakonec překlápí z hodnoty 0 na hodnotu 1.

Matematicky tento výpočet cílových pozic lze zapsat pomocí následující rovnice:

$$p_i = h_i(x) \pmod{m} \quad \text{pro } 1 \leq i \leq k, \quad (3.1)$$

Operaci vkládání prvku z informatického pohledu výborně reprezentuje algoritmus 1:

Algoritmus 1: Vložení prvku x do Bloomova filtru

Vstup: Prvek x , bitové pole B o velikosti m , množina hashovacích funkcí H o velikosti k

```

1 for  $i = 1$  to  $k$  do
2    $hash\_value \leftarrow h_i(x)$ 
3    $position \leftarrow hash\_value \pmod{m}$ 
4    $B[position] \leftarrow 1$ 
```

Důležitou klíčovou vlastností, kterou je potřeba zdůraznit, je pokud po aplikaci výpočtu operací modulo je ukázáno na bit, který byl již dříve nastaven na hodnotu jedna pomocí jiného vloženého prvku. Jeho hodnota zůstává nastavena na hodnotu jedna i nadále. Standardní Bloomův filtr je z podstaty věci navržen pouze pro přidávání záznamů. Umí bity překlápět z hodnoty nula na hodnotu jedna, nikoliv naopak. Z tohoto faktu přímo vyplývá, že prvky nelze odstraňovat. Kdybychom se pokusili otisk prvku z Bloomova filtru odstranit a bity, pod kterými byl prvek zahashován, vrátili na hodnotu nula, existovala by pravděpodobnost, že se nechtěně poškodí a zároveň zneplatní i otisk jiného prvku, který daný bit rovněž využívá.

3.2.2 Operace dotazování (vyhledávání)

Silnou stránkou této datové struktury je především její dotazovací mechanismus, který ve své podstatě jen zrcadlí proces vkládání. Testování přítomnosti hledaného prvku x , spočívá opět ve výpočtu hashe pomocí sady k hashovacích funkcí, kterou jsme na prvky použili, když se do filtru prvky vkládaly. Čímž získáme k indexových pozic v bitovém poli.

Algoritmus následně zkontroluje hodnoty na všech těchto pozicích:

- **Bezesporná absence:** Pokud je být *jen jediný* z testovaných bitů roven nule, znamená to, že otisk prvku není kompletní. Systém tak může s naprostou jistotou prohlásit, že prvek x se v množině nenachází. Právě zde vzniká ona garance nulového výskytu falešně negativních výsledků.
- **Nejistá přítomnost:** Jestliže jsou při kontrole *všechny* bity nastaveny na hodnotu jedna, algoritmus z tohoto výsledku odvodí, že prvek v množině s největší pravděpodobností je obsažen. Mohl do množiny být skutečně dříve vložen, nebo se jedná o falešně pozitivní výsledek. Stav, kdy k bitů nastavených na hodnotu jedna, nezávisle na sobě, nastavila kombinace jiných vložených prvků.

Rozhodování o přítomnosti prvku ze softwarového hlediska lze zapsat následujícím algoritmem 2.

Algoritmus 2: Dotaz na přítomnost prvku x v Bloomově filtru

Vstup: Prvek x , bitové pole B o velikosti m , množina hashovacích funkcí H o velikosti k

Výstup: True (pravděpodobně přítomen) nebo False (definitivně nepřítomen)

```
1 for  $i = 1$  to  $k$  do
2    $hash\_value \leftarrow h_i(x)$ 
3    $position \leftarrow hash\_value \pmod{m}$ 
4   if  $B[position] == 0$  then
5     return False
6 return True
```

3.3 Matematický model pravděpodobnosti falešně pozitivního výsledku

Využitelnost Bloomova filtru v praxi silně závisí na schopnosti predikace a kontroly chybovosti, která u filtru nastane. Ačkoliv s původním konceptem přišel Burton Bloom, o matematický model tak, jak ho známe dnes, se zasloužili až výzkumníci Broder a Mitzenmacher [5] počátkem nultých let. Jejich modely nám umožňují spočítat přesné riziko, že nám filtr ohlásí přítomnost prvku, který v něm ve skutečnosti vůbec není.

Když do filtru vkládáme konkrétní prvek, musí projít přes celou sérii k nezávislých hashovacích funkcí. Šance, že bit, který monitorujeme, zůstane nenastaven (s hodnotou 0), se tím umocňuje na $(1 - 1/m)^k$.

V praxi ale do filtru vkládáme mnohonásobně více prvků, celý dataset o velikosti n . Tehdy systém provede celkem $k \cdot n$ nezávislých hashovacích operací. Pravděpodobnost (kterou označíme p), že konkrétní bit zůstane i po dokončení všech zápisů stále na nule, vyjadřuje následující rovnice:

$$p = \left(1 - \frac{1}{m}\right)^{kn} \quad (3.2)$$

Avšak pokud bychom se pokusili tento vzorec přímo naprogramovat pro reálné systémy, kde se velikost pole m pohybuje v řádech stovek miliónů, narazili bychom na problém s hardwarovými limity při počítání s plovoucí desetinnou čárkou. Matematicky si ale můžeme pomoci limitou pro přirozený logaritmus.

$$\lim_{x \rightarrow \infty} \left(1 - \frac{1}{x}\right)^x = \frac{1}{e} \quad (3.3)$$

Pomocí substituce získáme výpočetně zvládnutelnou asymptotickou aproximaci:

$$p \approx e^{-\frac{kn}{m}} \quad (3.4)$$

Z tohoto vztahu vyplývá, že pravděpodobnost p_1 , aby byl bit překlopen na hodnotu jedna, získáme pouhým odečtením od celku:

$$p_1 = 1 - p \approx 1 - e^{-\frac{kn}{m}} \quad (3.5)$$

Nyní se dostáváme k jádru problému. Kdy přesně dojde k falešně pozitivnímu výsledku? Stane se tak v okamžiku, kdy testujeme prvek, který do filtru nikdy nebyl vložen, ale všech k hashovacích funkcí ukáže na bity, které byly nastaveny jinými prvky. Za předpokladu, že jsou jednotlivé bity nastavovány nezávisle na sobě, je celková chybovost označená ε a vypočtena v rovnici:

$$\varepsilon = (p_1)^k \approx \left(1 - e^{-\frac{kn}{m}}\right)^k \quad (3.6)$$

3.4 Optimalizace parametrů filtru

Podle vzorce (3.6) lze poukázat na velmi zajímavý konstrukční paradox. Pokud při návrhu filtru je zvolen příliš nízký počet hashovacích funkcí (malé k), otisky jednotlivých prvků nebudou dostatečně unikátní. Nezabránilo by se tak shlukování. Pokud ale bude hashovacích funkcí příliš mnoho, bitové pole se zaplní jedničkami velmi rychle. Tento stav je obecně nazván saturací filtru [13] a vede k drastickému a nekontrolovatelnému nárůstu kolizí.

K nalezení optimálního počtu hashovacích funkcí filtru vede upravený vzorec pro ε (3.6). Pokud se rovnice zderivuje podle proměnné k a výsledek položí roven nule, výsledkem je lokální minimum

chybovosti:

$$k_{optimal} = \frac{m}{n} \ln 2 \approx 0,693 \frac{m}{n} \quad (3.7)$$

Tento vzorec dokazuje jednu z klíčových vlastností Bloomových filtrů. Pokud tento ideální počet funkcí $k_{optimal}$ dosadíme zpět do vzorce pro pravděpodobnost nastaveného bitu p_1 (3.5), zjistíme, že $p_1 = 1 - e^{-\ln 2}$, což se rovná přesně $1/2$. Z tohoto výsledku vyplývá, že Bloomovy filtry fungují nejlépe a nejefektivněji, když je přesně polovina jeho bitů v bitovém poli naplněna jedničkami a polovina naplněna nulami. Pokud je pole prázdnější, plýtváme operační pamětí. Pokud je plnější, struktura začíná selhávat.

Jestliže je předem známa hodnota $k_{optimal}$, kolik prvků se zhruba do filtru bude vkládat n a jakou maximální chybovost by měl filtr tolerovat (například $\varepsilon = 0,01$ pro jedno procento chybných výsledků), je možné vypočítat jak velká paměť se pro daný Bloomův filtr musí alokovat. Izolováním proměnné m a dosazením do rovnic získáme následující vzorec pro požadovanou kapacitu bitového pole:

$$m = -\frac{n \ln \varepsilon}{(\ln 2)^2} \quad (3.8)$$

Tato rovnice mimo jiné dokazuje, že paměťová náročnost filtru roste lineárně s počtem prvků, bez ohledu na to, jak velká jsou vkládaná data. Aplikaci vzorce lze ukázat na reálném příkladu (3.9). Pro jeden milion prvků ($n = 1\,000\,000$) a zvolenou maximální 1 % chybovost získáme rovnici pro velikost paměti, která je potřeba alokovat pro celé bitové pole Bloomova filtru:

$$m = -\frac{1\,000\,000 \cdot \ln(0,01)}{(\ln 2)^2} \approx 9\,585\,058 \text{ bitů} \quad (3.9)$$

Celé bitové pole filtru tak zabere zanedbatelných $\approx 1,19$ MB. Když tento výsledek podělíme počtem prvků (v uvedeném příkladu (3.10) je to $1\,000\,000$), výsledná paměť, která je potřeba alokovat pro jediný prvek je rovna 9,6 bitů. Zda je tímto vkládaným prvkem obyčejné čtyřbajtové ID, nebo URL adresa s parametry, která je 2000 znaků dlouhá, nehraje při této alokaci paměti žádnou roli. Pokud bychom počítali kompletní architektonický návrh pro příklad výše, musíme ještě dopočítat ideální počet hashovacích funkcí pomocí rovnice (3.10).

$$k_{optimal} = \frac{9\,585\,058}{1\,000\,000} \ln 2 \approx 6,64 \quad (3.10)$$

Jelikož hashování nelze provést na zlomek pokusů, musíme výsledek matematicky zaokrouhlit a v tomto konkrétním příkladu použijeme přesně $k = 7$ hashovacích funkcí.

3.5 Analýza výhod a omezení

Využití pravděpodobnostních struktur v produkčním softwaru je o hledání kompromisů. Je to vědomá výměna, při které se obměňuje stoprocentní deterministická jistota za výkon, který by jinak

nebylo možné získat. Aby tato volba šla obhájit, musíme se objektivně podívat na silné i slabé stránky této technologie.

3.5.1 Klíčové benefity struktury

- Extrémní komprese prostoru: Obrovská redukce paměťové stopy je tím hlavním důvodem, proč Bloomovy filtry existují. Výhodu lze vidět na příkladu proxy serveru, který potřebuje v paměti držet černou listinu o velikosti deseti milionů škodlivých URL adres [5].

Pokud by pro uložení byla použita klasická deterministická *hash mapa* na 64bitové architektuře, musí se mimo samotná data počítat i s režii nad datovou strukturou. Průměrná délka jedné URL adresy se v literatuře uvažuje okolo 64 bajtů [5]. Pokud se k této hodnotě přičte paměťová režie samotného objektu (hlavička objektu a délka řetězce) a minimálně 8 bajtů pro ukazatel, jakožto 64bitový ukazatel. Celková velikost jednoho záznamu tak přibližně bude činit 100 bajtů. Celková paměťová náročnost ($Mem_{hashmap}$) pro deset milionů záznamů je dána vztahem:

$$Mem_{hashmap} = n \cdot 100 = 10\,000\,000 \cdot 100\text{ B} = 1\,000\,000\,000\text{ B} \approx 1\text{ GB} \quad (3.11)$$

Oproti tomu Bloomův filtr konfigurovaný pro naprosto stejný objem dat ($n = 10\,000\,000$) se stanovenou chybovostí $\varepsilon = 0,01$ vyžaduje paměť určenou optimalizačním vzorcem:

$$m = -\frac{10\,000\,000 \cdot \ln(0,01)}{(\ln 2)^2} \approx 95\,850\,584\text{ bitů} \quad (3.12)$$

Z čehož vyplývá, že celkový objem paměti, který Bloomův filtr zabere je $\approx 11,43\text{ MB}$.

Matematický model tak dokazuje, že úspora místa je bezmála 98 %. Zatímco gigabajtová tabulka si vynutí přístup do pomalejší RAM, Bloomův filtr o velikosti 11 MB se vejde do L2 nebo L3 cache procesoru.

- Garantovaná rychlost $\mathcal{O}(k)$: Když uvedeme Bloomovy filtry do kontrastu s klasickými vyhledávacími stromy, které mají časovou složitost $\mathcal{O}(\log n)$, časová složitost Bloomova filtru je plně závislá pouze na počtu hashovacích funkcí (k), které musíme spočítat. V praxi je k obvykle velmi malé konstantní číslo, tudíž je rychlost dotazu konstantní a stabilní, zcela nezávislá na počtu prvků, které filtr obsahuje.
- Přirozené šifrování dat: Tím, že do paměti neukládáme původní data, ale pouze pole jedniček a nul rozvržených pomocí jednosměrných hashovacích funkcí, seznam původních prvků z bitového pole není možné zpětně zrekonstruovat.

3.5.2 Limity a překážky struktury

- Neschopnost mazání data: Jakmile do filtru prvek vložíme a přepíšeme bity v bitovém poli z nuly na jedničku, nemůžeme tuto akci vzít zpět. Představa, že prvek se smaže pouze překlopením bitů, kde otisk daného prvku je roven jedničce, je nesprávná. Mohly by být překlopeny i bity, které využívá jiný prvek. Tím bychom z filtru smazali i tento druhý prvek a porušili bychom základní pravidlo Bloomova filtru o nevyskytujících se falešně negativních výsledcích. Tento problém v praxi řeší upravená varianta filtru zvaná *Počítací Bloomův filtr* [2].
- Saturace kapacity: Pole o velikosti m se v operační paměti alokuje staticky ihned na začátku. Pokud se v systému podcení objem dat a do filtru se vloží násobně více dat než pro kolik byl filtr navržen, pole se přesytí jedničkami a chybovost ε okamžitě vzroste k neakceptovatelným hodnotám. Strukturu filtru nejde snadno dynamicky zvětšit. Pokud by opravdu bylo potřeba zvětšit kapacitu Bloomova filtru, musela by se vytvořit nová instance a všechny prvky by musely být znovu zahashovány. Což je problém ve chvíli, kdy stará data již nemáme k dispozici.

Kapitola 4

Vývoj a varianty Bloomova filtru

Klasický Bloomův filtr je sice velkým přínosem ve své prostorové jednoduchosti, ale jeho architektonická nepřizpůsobivost s sebou nese spoustu problémů. Jak jsme již rozebrali v závěru předchozí kapitoly, systémy často potřebují stará data nejen zapisovat, ale i promazávat. Stejně tak je v praxi jen zřídkakdy možné předem s naprostou přesností predikovat, jak velká vkládaná data nakonec budou.

Vývojáři si těchto limitů byli vědomi a postupně navrhli řadu strukturálních modifikací, které původní koncept rozšiřují. V této kapitole rozebereme ty nejzásadnější z nich, které tvoří teoretický základ pro softwarovou implementaci v praktické části této práce.

4.1 Počítací Bloomův filtr (Counting Bloom Filter)

Řešení problému nemožnosti z filtru prvky odebírat představili Fan, Cao, Almeida a Broder ve svém článku o distribuovaných webových mezipamětech [2]. Jejich inovace, nazvaná Počítací Bloomův filtr (*Counting Bloom Filter*, zkráceně CBF), zasahuje přímo do samotné anatomie bitového pole. Místo pole primitivních bitů, které nesou pouze informaci 0 nebo 1, využívá CBF pole malých celočíselných čítačů.

4.1.1 Mechanismus inkrementace a mazání

Přechod z bitů na čítače mírně mění základní operace struktury. Při vkládání prvku algoritmus vygeneruje k hashovacích pozic, ale místo dřívějšího nastavení bitu na jedničku vezme aktuální hodnotu na daném indexu a inkrementuje ji o 1.

Při dotazování na přítomnost prvku se systém podívá na vypočítané pozice a prvek označí za přítomný pouze tehdy, pokud je hodnota všech dotčených čítačů ostře větší než nula. Výhoda fungování CBF se pak naplno projevuje při operaci mazání. Pokud chce systém prvek odstranit, jednoduše znovu spočítá jeho hashovací pozice a hodnoty příslušných čítačů naopak dekrementuje.

Tím se bezpečně vyřeší nebezpečí překrývajících se bitů v poli, aniž by se narušily otisky ostatních prvků, které daný index sdílejí.

4.1.2 Matematické zdůvodnění velikosti čítače

Jelikož již v anatomii filtru není použito pouze bitové pole, je nutné zvolit jiný datový typ, který bude v bitovém poli uložen. Kdybychom použili standardní *32-bitový integer*, původní výhoda paměťové komprese by se okamžitě vytratila a struktura by byla zhruba dvaatřicetkrát větší než základní filtr.

Fan a jeho tým ve své práci poskytli exaktní statistický důkaz [2]. Pokud předpokládáme, že je filtr správně dimenzován a používá optimální počet hashovacích funkcí, řídí se pravděpodobnost toho, že konkrétní čítač dosáhne hodnoty z , klasickým binomickým rozdělením. Autoři matematicky dokázali, že šance na extrémní navýšení jednoho čítače je ostře ohraničena vztahem z rovnice:

$$P(\max Z \geq z) \leq m \binom{nk}{z} \left(\frac{1}{m}\right)^z \quad (4.1)$$

Vyhodnocením tohoto modelu se dospělo k fascinujícímu závěru. Pravděpodobnost, že by jeden a ten samý index v bitovém poli byl hashovacími funkcemi vybrán více než patnáctkrát, je za normálních provozních podmínek velmi malá – konkrétně $1,37 \times 10^{-15}$ [2].

Desítkovou hodnotu 15 lze v binární soustavě zapsat pomocí přesně 4 bitů, proto se čtyřbitový čítač stal standardem. I přes to, že je CBF paměťově zhruba čtyřikrát náročnější než jeho standardní předchůdce, stále zůstává o několik řádů úspornější než klasické deterministické struktury.

4.1.3 Zranitelnost fantomového mazání a přetečení

Největší hrozbou u CBF je tzv. fantomové mazání. Systém nikdy nesmí připustit smazání prvku, který do něj předtím reálně nebyl vložen. Pokud by k tomu došlo (například kvůli neopatrnému zpracování falešně pozitivního výsledku), aplikace by snížila čítače, které ve skutečnosti patřily legitimním prvkům. Tím by se struktura nenávratně znehodnotila a byla by zde šance dostat falešně negativní výsledek.

Druhým rizikem je samotné přetečení čítače. Přestože je pravděpodobnost překročení hodnoty 15 minimální, v dlouhodobě běžících systémech k ní teoreticky dojít může. Aby se zabránilo tomu, že čítač „přeteče“ zpět na nulu (čímž by definitivně smazal data), implementuje se technika zvaná *pinning* [2]. Jakmile čítač dosáhne svého maxima, trvale se uzamkne a přestane reagovat na další inkrementace i dekrementace. Trvale se tak chová jako bit překlopený na hodnotu jedna ve standardním filtru.

4.2 Škálovatelné Bloomovy filtry (Scalable Bloom Filter)

Druhým fatálním nedostatkem standardního Bloomova filtru je nutnost znát předpokládanou kapacitu n ještě předtím, než program vůbec spustíme. Jakmile bitové pole jednou alokujeme v RAM, stává se z něj fixní blok. Pokud do něj následně vložíme násobek dat, než na kolik byl dimenzován, stane se z něj přesycené pole jedniček, jehož chybovost se limitně blíží ke 100 %.

Tento problém zkusil vyřešit tým Paula Almeidy v roce 2007, když publikoval koncept Škálovatelných Bloomových filtrů (SBF) [3]. Jde o elegantní softwarový návrhový vzor, který umožňuje struktuře dynamicky a prakticky donekonečna růst.

4.2.1 Architektura zřetězení

Škálovatelný filtr vlastně není jen jedna datová struktura, ale chytrý manažer, který spravuje celé pole klasických Bloomových filtrů. Celý proces začíná alokací jednoho úvodního filtru F_0 , kterému je přiřazena určitá výchozí kapacita a maximální možná tolerance chyb ε_0 .

Systém začne vkládat data do tohoto prvního filtru. Zároveň ale vnitřní logika neustále hlídá míru jeho zaplnění. Jak víme již z předchozí kapitoly 3.3, filtr ztrácí efektivitu, jakmile překročí hranici zaplněnosti 50 %. Jakmile F_0 tohoto bodu nasycení dosáhne, manažer do něj přestane zapisovat nová data a okamžitě vedle něj vytvoří nový filtr, který označíme F_1 . Aby nemusel tvořit nové filtry příliš často, nově vytvořený filtr má obvykle dvojnásobnou kapacitu oproti tomu předchozímu. Nová data se tak začnou vkládat do F_1 . Takto může řetězec růst do nekonečna.

4.2.2 Matematické ošetření chybovosti

Skrývá se v tom ovšem jeden drobný matematický háček. Při dotazování na přítomnost prvku musíme prohledat všechny existující filtry v řetězci. Pokud alespoň jeden z nich odpoví pozitivně, výsledek se vrací jako *True*. Každý z těchto filtrů má ale svou vlastní chybovost. Kdybychom vedle sebe donekonečna řetězili filtry s chybovostí 1 %, celková chybovost celého tohoto systému by se postupně sčítala a nakonec by opět dosáhla nepřijatelných hodnot.

Almeida tento problém ve své práci vyřešil pomocí geometrické posloupnosti. Dokázal, že aby globální chybovost řetězce zůstala omezena na počáteční hodnotu, musí mít každý nově vytvořený filtr mírně nižší toleranci k chybám než měl filtr před ním. Pokud si jako zpřísnující faktor zvolíme proměnnou r (kde $r < 1$, např. 0,5), bude maximální povolená chybovost pro každý další i -tý filtr omezena vzorcem:

$$\varepsilon_i = \varepsilon_0 \cdot r^i \quad (4.2)$$

Výměnou za tuto dynamickou rozšiřitelnost je bohužel ztráta konstantní časové složitosti. Protože dotaz musí v nejhorším případě projít úplně všechny vytvořené filtry, čas hledání degraduje z původního $\mathcal{O}(k)$ na $\mathcal{O}(k \cdot L)$, kde L reprezentuje aktuální počet vytvořených filtrů [3].

Kapitola 5

Praktická část: Návrh a implementace

Hlavním cílem praktické části je navrhnout, implementovat a následně testy vyhodnotit knihovnu pravděpodobnostních datových struktur pro platformu .NET. Výsledný kód je navržen tak, aby mohl být distribuován a snadno integrován jako standardní NuGet balík.

Tato kapitola detailně mapuje celý životní cyklus vývoje knihovny: od volby metodiky a formální analýzy požadavků, přes definici architektury zajišťující bezpečný běh ve vícevláknovém prostředí, až po zápis jednotlivých operací v jazyce C#. Během vývoje byl kladen důraz na čistotu kódu a čitelnost. Architektura i implementace dodržují oficiální jmenné konvence pro jazyk C# [14]. Průběžné verzování a zálohování projektu probíhalo pomocí systému Git, z toho hlavní repozitář je hostován na platformě GitHub. Součástí repozitáře je projektová dokumentace ve formě *GitHub Wiki*, která slouží jako uživatelská příručka pro práci s knihovnou.

5.1 Analýza požadavků a metodika vývoje

Principy Bloomových filtrů, co se týče pravidel fungování, jsou z definice striktně deterministické. Jejich chování, vstupy, výstupy i matematické limity jsou pevně dány teorií, kterou jsme popsali v předchozích kapitolách 2 a 3. Neexistuje zde žádný koncový uživatel, který by v průběhu vývoje měnil, jak má vypadat uživatelské rozhraní.

Z tohoto důvodu byl pro řízení projektu zvolen klasický lineární vodopádový model. Jeho sekvenci přístup naprosto přesně splňuje strukturu této bakalářské práce. Každá dokončená fáze projektu sloužila jako pevný a již neměnný základ pro fázi následující.

5.1.1 Charakteristika požadavků pomocí modelu FURPS

Pro zajištění, že výsledná knihovna bude použitelná v produkčním nasazení, rozhodl jsem se použít formální specifikaci požadavků pomocí standardu FURPS ¹.

¹Model FURPS – softwarové požadavky do pěti kategorií podle písmen anglických slov: *Functionality* (funkčnost), *Usability* (použitelnost), *Reliability* (spolehlivost), *Performance* (výkonnost) a *Supportability* (podporovatelnost).

- **Funkční požadavky:** Jádrem celého balíku musí být bezchybná implementace tří klíčových datových struktur:

- Klasický Bloomův filtr: Musí podporovat základní operace vkládání prvků a dotazování na členství s volitelnou šancí na falešně negativní výsledek.
- Počítací filtr (CBF): Rozšiřuje funkčnost o možnost bezpečně odebírat již vložené prvky. Návrh počítá s implementací osmibitových čítačů pro minimalizaci paměťové stopy.
- Škálovatelný filtr (SBF): Musí zajistit dynamický růst kapacity struktury. Při naplnění primárního bitového pole musí systém automaticky alokovat nové pole s exponenciálně přísnější tolerancí vůči chybám a dvojnásobnou kapacitou.

Dalším funkčním požadavkem je také nástroj pro tvorbu konfigurace (*Builder*), který vývojáře odstíní od složité matematiky. Uživatel by měl zadat pouze očekávaný počet prvků (n) a tolerovanou chybovost (ε), přičemž systém na pozadí sám aplikuje odvozené vzorce a dopočítá optimální velikost pole (m) a počet hashovacích funkcí (k).

- **Výkonnostní požadavky:** Protože jsou tyto struktury určeny pro nasazení na úrovni databází či síťových routerů, je výkon naprosto kritický. Obě primární operace (vlození i vyhledání) musí bezvýhradně splňovat konstantní časovou složitost $\mathcal{O}(1)$ vůči počtu již vložených prvků (neboli čas $\mathcal{O}(k)$ závislý pouze na počtu hashovacích iterací). Návrh se musí striktně vyvarovat zbytečných alokací paměti na haldě, aby nedocházelo k častému spouštění Garbage Collectoru.
- **Spolehlivost a bezpečnost:** Předpokládá se, že do knihovny bude přistupovat obrovské množství vláken současně. Architektura proto musí být od počátku koncipována jako *thread-safe*.
- **Použitelnost a rozšiřitelnost:** Z uživatelského hlediska musí knihovna plně respektovat konvence a standardy platformy .NET. Tím je myšleno především využití generických typů (`<T>`), díky nimž bude možné do filtru transparentně vkládat jak primitivní datové typy (čísla, řetězce), tak komplexní uživatelské objekty. Balíček musí být kompilován vůči standardu .NET Standard nebo moderním .NET specifikacím, čímž se zaručí jeho přenositelnost napříč operačními systémy (Windows, Linux) bez jakýchkoliv těžkých externích závislostí. Celý projekt bude obsahovat XML dokumentaci a bude hostován na platformě GitHub včetně Wiki stránek.

5.2 Architektonický návrh a objektový model

Aby byla knihovna snadno udržitelná, testovatelná a rozšiřitelná, musí její vnitřní architektura striktně dodržovat principy oddělení odpovědností jednotlivých částí. Architektonický návrh tohoto balíku byl proto od samého počátku koncipován modulárně, což se přímo odráží v hierarchii jmenných prostorů a definovaných rozhraní.

5.2.1 Dekompozice systému a rozhraní

Z hlediska celkové organizace zdrojových kódů byl projekt rozdělen na tři zcela izolované části. Základ tvoří samotná produkční knihovna `BloomFilter`. Tu doplňují dva podpůrné projekty: `BloomFilter.Tests` (obsahující sadu jednotkových testů pro ověření spolehlivosti struktury) a `BloomFilter.Benchmarks` (obsahující zátěžové a výkonnostní testy). Toto striktní oddělení zaručuje, že finální NuGet balíček knihovny neobsahuje žádný redundantní testovací kód, který by zbytečně navyšoval jeho velikost.

Úplným základem knihovny je tedy jmenný prostor `BloomFilter`, který sdružuje samotné datové struktury. Abychom vývojářům poskytli jednotný a předvídatelný předpis pro práci s filtry, byla veškerá funkcionalita přenesena do abstraktního a generického rozhraní, konkrétně `IBloomFilter<T>` a jeho specializovanou variantu `ICountingBloomFilter<T>`. Využití generik (`<T>`) na úrovni rozhraní umožňuje do filtru transparentně vkládat jakékoliv datové typy, od prostých řetězců až po komplexní doménové objekty, přičemž se systém postará o jejich převod na bajtovou reprezentaci interně.

Kromě samotných datových struktur je projekt rozdělen do dalších, specializovaných modulů, které je možné vidět na třídním diagramu A.1:

- Modul Hashing (`BloomFilter.Hashing`): Zcela izoluje logiku výpočtu hashovaných indexů. Definuje rozhraní `IHashFunction` a obsahuje implementace rychlých nekryptografických algoritmů (jako jsou *MurmurHash3* nebo *XXHash*). Architektura navíc zavádí tzv. `DoubleHashWrapper`, který slouží jako překladová vrstva. Tento návrhový vzor (*Wrapper*) [15] efektivně řeší problém generování k nezávislých pozic ze dvou základních hashů, čímž šetří výpočetní výkon.
- Modul BloomMath (`BloomFilter.BloomMath`): Zapouzdřuje veškeré teoretické vzorce odvozené v kapitole 3.3. Oddělení matematiky do samostatného modulu zaručuje, že samotná třída filtru nemusí řešit výpočty logaritmů a limit, ale pouze konzumuje hotové parametry.
- Modul IO (`BloomFilter.IO`): Poskytuje nástroje pro serializaci a export filtrů, což slouží pro jejich přenos po síti nebo ukládání na disk.
- Modul Utils (`BloomFilter.Utils`): Obsahuje podpůrné nástroje, které abstrahují operace mimo hlavní doménovou logiku. Klíčovou komponentou je statická třída `BitArrayExtension`, která zajišťuje bezpečné a efektivní převody mezi strukturou `BitArray` a polem bajtů (`byte[]`).

5.2.2 Identifikace případů užití

Návrh API a vnitřní struktury vycházejí z reálných scénářů, ve kterých se pravděpodobnostní filtry typicky nasazují. Lze tedy definovat dva primární případy užití:

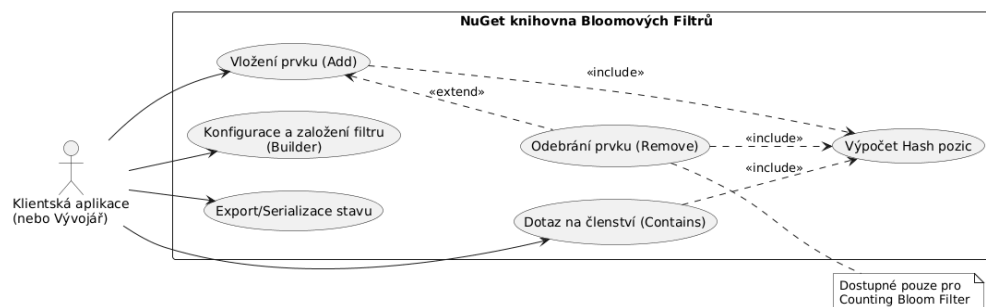
1. Ochrana před drahými I/O operacemi: Vývojář začleňuje knihovnu do databázového engine nebo webové mezipaměti. Aplikace filtruje každý příchozí dotaz metodou `Contains`. Zde architektura nesmí uživatele zatěžovat manuálním výpočtem kapacity. Návrh proto využívá návrhový vzor *Builder* (`BloomFilterBuilder`), přes který vývojář deklaruje pouze vlastní požadavky (např. očekávám milion prvků a toleruji chybovost 1 %) a systém mu vrátí hotovou, správně dimenzovanou instanci.
2. Distribuované ověřování stavu: Architektura počítá se scénářem, kdy jeden server filtr naplní daty a následně jej potřebuje distribuovat na koncová zařízení. Pro tento případ užití jsou v modulu IO připraveny serializační třídy, které umožňují překlopit vnitřní stav filtru (samotné bitové pole a konfiguraci hashování) do definovaného datového proudu.

Dále, na základě analýzy požadavků FURPS a výše zmíněných scénářů byly identifikovány hlavní interakce aktéra se systémem, které můžeme také vidět na diagramu 5.1:

- **Konfigurace a založení filtru:** Aktér inicializuje strukturu přes *Builder* definováním očekávané kapacity a tolerované chybovosti. Tento případ užití interně využívá modul `BloomMath` pro výpočet optimálních parametrů.
- **Vložení prvku:** Základní operace pro sbírání dat. Aktér předává generický objekt, který je systémem hashován a zapsán do struktury.
- **Dotaz na členství:** Nejčastější operace omezující zbytečné databázové I/O dotazy. Aktér se dotazuje na existenci prvku a systém vrací pravděpodobnostní odpověď (zcela jistě ne nebo pravděpodobně ano).
- **Odebrání prvku:** Specifický případ užití, který rozšiřuje základní manipulaci s daty. Je dostupný výhradně při založení Počítacího Bloomova filtru (CBF).
- **Export a serializace stavu:** Operace navržená přímo pro scénář distribuovaného ověřování. Aktér vyžádá překlopení interního stavu bitového pole do binárního formátu pro přenos po síti nebo uložení na disk.

5.2.3 Strategie pro vícevláknové použití

Zcela kritickým bodem návrhu bylo zajištění bezpečnosti ve vícevláknovém prostředí (*Thread-safety*). Kolekce v platformě .NET, a to včetně třídy `BitArray`, která tvoří hlavní část našeho filtru, nejsou ze své podstaty stavěné na souběžný přístup. Pokud by se dvě nezávislá vlákna pokusila ve stejný okamžik zapsat hodnotu do bitového pole nebo by jedno vlákno zapisovalo a druhé četlo, došlo by k porušení integrity paměti a ke znehodnocení datové struktury. K vyřešení tohoto



Obrázek 5.1: Use Case diagram znázorňující interakci klientské aplikace s API knihovny

problému byla v této práci zvolena strategie striktního exkluzivního zámku pomocí standardního konstruktů `lock`.

Tento architektonický přístup funguje na principu vzájemného vyloučení. Kdykoliv vlákno vstoupí do metody `Add` nebo `Contains`, uzamkne pro sebe vyhrazený objekt (`_lock`). Jakékoliv další vlákno, které by se v tu chvíli pokusilo o interakci s filtrem, je operačním systémem pozastaveno a zařazeno do fronty, dokud první vlákno svou práci nedokončí.

5.3 Implementace struktury

V této kapitole je rozebírána konkrétní programová implementace navržených struktur v jazyce C#. Zaměřuje se na překlad teoretických matematických modelů do efektivního kódu, správu paměti a architektonická rozhodnutí.

5.3.1 Vnitřní implementace filtru a reprezentace bitového pole

Celá knihovna je postavena na třídě `BloomFilter<T>`, která představuje klasickou variantu struktury implementující rozhraní `IBloomFilter<T>`.

Zcela klíčovým a nelehkým rozhodnutím, které definuje výkon a paměťovou složitost celé knihovny, byla volba reprezentace samotného bitového pole. Podle zadání byly zvažovány tři rozdílné přístupy:

1. Vlastní nízkoúrovňová implementace: Vytvoření pole celých čísel (např. `ulong[]`) a manuální správa bitů pomocí bitových posunů a bitových masek. Tento přístup nabízí absolutní kontrolu nad pamětí, avšak přináší riziko chyb v indexaci a výrazně zvyšuje složitost zdrojového kódu.
2. Knihovny třetích stran: Využití již optimalizovaných struktur z existujících open-source balíčků (např. `FastBitArray`). Zde však existuje riziko externích závislostí, které zbytečně nabízejí vlastnosti, jež u Bloomova filtru nevyužijeme. Navíc by velikost naší cílové knihovny bezdůvodně vzrostla.
3. Nativní struktury platformy .NET: Využití vestavěné třídy `BitArray` ze jmenného prostoru `System.Collections`.

Po provedení analýzy poměru mezi složitostí údržby a reálným výkonem byla zvolena třetí varianta — nativní `BitArray`. Toto rozhodnutí je kritické z hlediska paměťové efektivity. Standardní pole booleovských hodnot (`bool[]`) alokuje v platformě .NET pro každý prvek celý jeden bajt (8 bitů), přestože nese informaci pouze o jednom bitu. Naproti tomu `BitArray` provádí interní optimalizaci a balí jednotlivé bity těsně za sebe do pole 32bitových celých čísel (`int[]`).

5.3.2 Návrhový vzor Builder a odstínění komplexity

Abychom zabránili chybné konfiguraci parametrů Bloomova filtru, je toto nastavení před uživateli skryto. Chybné nastavení by vedlo buď k plýtvání pamětí, nebo k neakceptovatelné míře falešně pozitivních výsledků. Proto architektura implementuje návrhový vzor *Builder* prostřednictvím třídy `BloomFilterBuilder<T>`. Ta poskytuje plynulé rozhraní, pomocí kterého uživatel deklaruje pouze své požadavky — očekávanou kapacitu (n) a tolerovanou chybovost (ε).

Složitý výpočet je před uživatelem zcela skryt a probíhá až v okamžiku volání finální metody `Build()`. Ta interně deleguje výpočet na statický modul `BloomMath`, jak demonstruje následující kód:

```
public IBloomFilter<T> Build()
{
    ValidateParameters();

    // Automatický výpočet optimálních parametrů (m, k)
    var (m, k) = BloomMath.BloomMath.CalculateOptimalParameters(_capacity,
        _falsePositiveRate);

    // Inicializace hashovacích funkcí
    var hashWrapper = CreateHashWrapper();

    return new BloomFilter<T>(m, k, _capacity, hashWrapper);
}
```

Zdrojový kód 5.1: Implementace metody `Build` delegující výpočet na modul `BloomMath`

Tento architektonický přístup zaručuje, že výsledná instance filtru je vždy matematicky optimální a její vnitřní stav bude validní, aniž by programátor musel znát teorii na pozadí.

5.3.3 Optimalizace otisků: Technika Double Hashing

Teoretický model Bloomova filtru předpokládá využití k vzájemně nezávislých hashovacích funkcí. V reálné softwarové implementaci by však neustálé inicializování a volání k odlišných hashova-

cích algoritmů (např. pro $k = 7$) představovaly neakceptovatelnou výpočetní zátěž, která by zcela degradovala propustnost filtru.

Tento problém řeší implementace techniky tzv. *Double Hashingu* [16], kterou pro potřeby Bloomových filtrů formálně definovali Kirsch a Mitzenmacher [16], která je zapouzdřena ve třídě `DoubleHashWrapper`. Algoritmus využívá pouze dvě rychlé, nekryptografické hashovací funkce (v našem případě *MurmurHash3* a *XXHash*, případně dvě instance stejné funkce s odlišným počátečním vektorem, tzv. *seedem*).

Z výstupů těchto dvou základních funkcí jsou následně pomocí lineární kombinace odvozeny všechny zbývající pozice podle matematického předpisu (5.1):

$$h_i(x) = (h_1(x) + i \cdot h_2(x)) \pmod{m}, \quad (5.1)$$

kde i nabývá hodnot od 0 do $k - 1$.

Následující ukázka kódu demonstruje tělo metody `GetHashPositions`, která tento algoritmus převádí do jazyka C#:

```
public int[] GetHashPositions(byte[] data, int k, int m)
{
    int[] positions = new int[k];
    uint hashA = _hash1.Hash(data);
    uint hashB = _hash2.Hash(data);

    // Ochrana proti degradaci na jedinou hashovací funkci
    if (hashB == 0)
    {
        hashB = 1;
    }

    for (int i = 0; i < k; i++)
    {
        // Lineární výpočet: hash_i(x) = (hashA(x) + i * hashB(x)) % m
        uint hash = hashA + ((uint)i * hashB);
        positions[i] = (int)(hash % (uint)m);
    }

    return positions;
}
```

Zdrojový kód 5.2: Lineární generování k pozic pomocí techniky Double Hashing

Praktickou aplikaci algoritmu lze také ukázat na konkrétním příkladu. Definujme si Bloomův filtr s následující konfigurací parametrů:

- Kapacita bitového pole $m = 20$. Indexy mohou nabývat hodnot v rozsahu 0 až 19.
- Počet hashovacích funkcí $k = 3$.
- Vkládaný prvek x , reprezentovaný konkrétním textovým řetězcem „*Petra Čechová*“.

V prvním kroku je vstupní prvek x zpracován dvěma nezávislými, nekryptografickými hashovacími funkcemi (např. MurmurHash3 společně s xxHash). Pro účely demonstrace předpokládáme následující výstupy těchto bázových funkcí:

- $h_1(x) = 42$
- $h_2(x) = 17$

Následně je aplikován přepis definovaný rovnicí (5.1) iterativním dosazením parametru $i \in \{0, 1, 2\}$. Horní mez odpovídá počtu $k - 1$, tímto získáme výsledné pozice v bitovém poli:

- Pro $i = 0$:
 $h_0(x) = (42 + 0 \cdot 17) \pmod{20} = 42 \pmod{20} = \mathbf{2}$
- Pro $i = 1$:
 $h_1(x) = (42 + 1 \cdot 17) \pmod{20} = 59 \pmod{20} = \mathbf{19}$
- Pro $i = 2$:
 $h_2(x) = (42 + 2 \cdot 17) \pmod{20} = 76 \pmod{20} = \mathbf{16}$

Algoritmus tedy jen se dvěma hashovacími funkcemi vygeneroval pro řetězec *Petra Čechová* indexy 2, 19 a 16. Na těchto vypočítaných pozicích dojde v bitovém poli k překlopení hodnoty z 0 na hodnotu 1. Tento postup zcela eliminuje nutnost implementace třetí a jakékoliv další nezávislé hashovací funkce.

5.3.4 Implementace Počítacího filtru

V Kapitole 4.1 se doporučuje pro strukturu CBF velikost čítače se čtyřbitovou kapacitou, což s vysokou pravděpodobností zabraňuje jejich přetečení. Platforma .NET však nativní čtyřbitový datový typ neobsahuje. Proto bylo pro návrh využito standardní pole bajtů (`byte[]`). Ačkoliv filtr spotřebuje dvojnásobek operační paměti oproti teoretickému optimu, získává maximální propustnost díky přirozenému zarovnání dat v paměti a využití nativních instrukcí procesoru.

Osmibitová kapacita navíc poskytuje obrovskou rezervu (maximální hodnota 255). I přesto musí být metody pro modifikaci stavu důsledně chráněny proti přetečení a podtečení, jak demonstruje následující zkrácený výpis klíčových operací:

```

// Předtím proběhlo hashování a výpočet pozic
foreach (int position in positions) {
    // Inkrementace pouze pokud nedošlo k dosažení maxima
    if (_counters[position] < byte.MaxValue) {
        _counters[position]++;
    }
}

// Předtím proběhlo hashování a výpočet pozic
foreach (int position in positions) {
    // Dekrementace pouze pokud je čítač kladný
    if (_counters[position] > 0) {
        _counters[position]--;
    }
}

```

Zdrojový kód 5.3: Ochrana čítačů při operacích Add a Remove

5.3.5 Implementace Škálovaného filtru

Pro nasazení v systémech, kde nemůžeme odhadnout velikost množiny dat, implementuje knihovna strukturu `ScalableBloomFilter<T>`. Tato struktura interně využívá návrhový vzor *Composite*. Navenek se tváří jako jediný filtr (implementuje jednotné rozhraní `IBloomFilter<T>`), ve skutečnosti však spravuje dynamický seznam instancí (`List<IBloomFilter<T>`).

```

private void AddNewFilter()
{
    // Vytvoření filtru s přísnějšími parametry pomocí Builderu
    IBloomFilter<T> newFilter = _builder
        .WithCapacity(_currentCapacity)
        .WithFalsePositiveRate(_currentFalsePositiveRate)
        .Build();

    _filters.Add(newFilter);
    _currentCapacity *= _scalingFactor;
    _currentFalsePositiveRate *= _tighteningRatio;
}

```

Zdrojový kód 5.4: Alokace nové vrstvy ve škálovatelném filtru

Kapitola 6

Benchmarking a experimenty

Ověření teoretických vlastností Bloomých filtrů v praxi také patří do vývoje. Cílem je prokázat matematickou správnost operací pomocí jednotkových testů a následně analyzovat paměťovou náročnost a propustnost pomocí profilovacích nástrojů *Benchmark*.

6.1 Metodika a prostředí testování

Pro provedení výkonnostních testů byl využit standard v jazyce C# **BenchmarkDotNet**. Tento nástroj automaticky řeší problémy spojené s během aplikací na platformě .NET, jako je optimalizace kódu za běhu a vliv garbage collectoru. Výsledky tak představují statisticky stabilní data očištěná o náhodné výkyvy operačního systému.

Tabulka 6.1: Hardwarové specifikace testovacích stanic

Zařízení	Procesor	L1 Data Cache	L2 Cache	L3 Cache
Pracovní zařízení 1	Intel Core i7-13700KF	8 × 48 kB	8 × 2048 kB	30 MB
Pracovní zařízení 2	AMD Ryzen 9 5950X	16 × 32 kB	16 × 512 kB	64 MB
Pracovní zařízení 3	Intel Core i7-1355U	2 × 48 kB	2 × 1280 kB	12 MB

Na všech testovacích stanicích uvedených v Tabulce 6.1 byl nainstalován framework .NET 8.0. Všechny testy byly spouštěny v režimu *Release*.

6.2 Jednotkové testování

Jelikož bylo nutné ověřit i logickou správnost implementace, byl za tímto účelem vytvořen testovací projekt využívající framework xUnit. Sada obsahuje celkem 50 automatizovaných testů, které pokrývají správnost výpočtů, správné hashování i různé stavy datových struktur.

Testovací scénáře jsou logicky rozděleny do několika kategorií:

- **Testy matematického modelu (MathTests):** Ověřují, že algoritmy pro výpočet optimální velikosti pole m a optimálního počtu hashovacích funkcí k vrací hodnoty odpovídající teoretickým vzorcům pro zadanou kapacitu a chybovost.
- **Testy hashovacích algoritmů (HashingTests):** Kontrolují správnost hashů funkcí MurmurHash3 a XXHash. Zvláštní důraz je kladen na `DoubleHashWrapper`, kde se ověřuje generování správného počtu indexů a striktní dodržení hranic alokovaného pole.
- **Ověření stavů datových struktur (BloomFilterTests a CountingFilterTests):** Testují správnou detekci falešné positivity. U CBF je navíc dedikovanými testy ověřena ochrana proti podtečení a přetečení.
- **Testy dynamického škálování (ScalableFilterTests):** Prokazují, že při vyčerpání kapacity dojde k bezpečné alokaci nového filtru a že dotazování správně prohledává celou hierarchii dříve vytvořených filtrů.
- **Testy serializace a exportování (SerializerTests):** Porovnávají výsledky serializace do binárních souborů a souborů s příponou `.json`.

6.3 Výkonnostní profilování (Benchmarks)

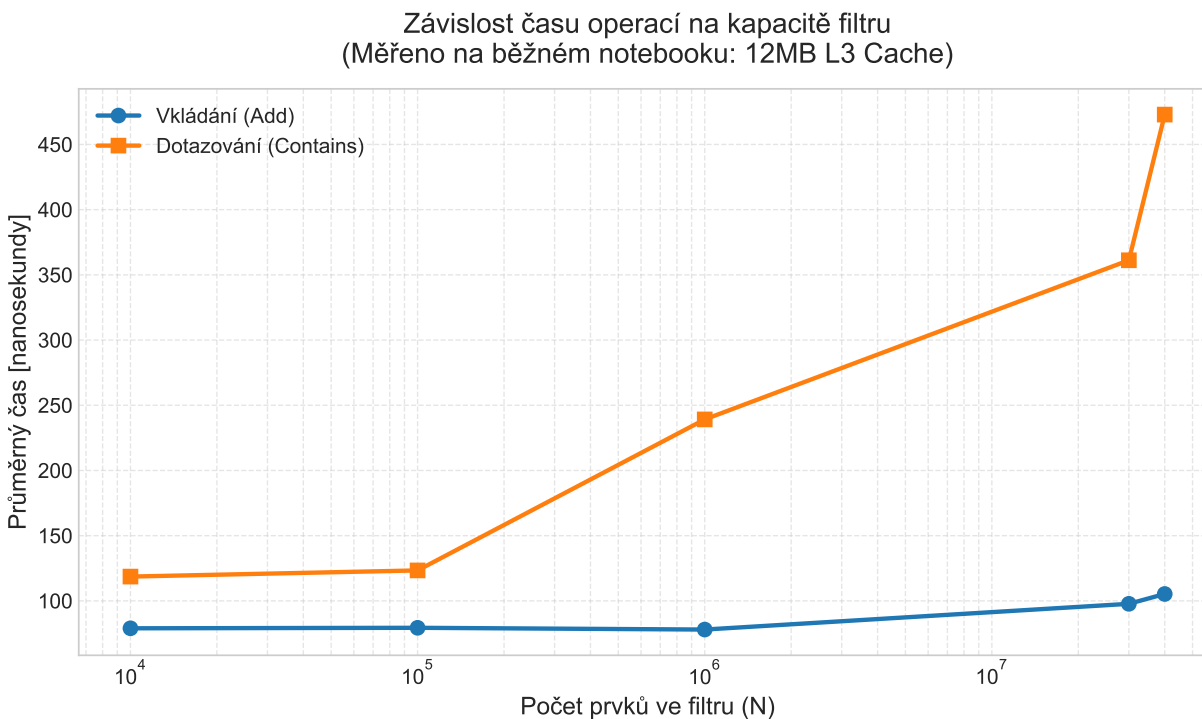
Fáze vyhodnocení výkonu knihovny se zaměřuje na propustnost a latenci samotných operací nad Bloomovými filtry. Měření na úrovni nanosekund odhaluje i interakci pravděpodobnostních struktur s architekturou moderních procesorů, konkrétně s hierarchií vyrovnávacích pamětí.

Výsledky v této podkapitole jsou získány nástrojem `BenchmarkDotNet` pro různé objemy dat a odlišné hardwarové platformy. Celé výsledky je možné vidět v Příloze A.

6.3.1 Důkaz konstantní složitosti a vliv L3 Cache

Základním teoretickým předpokladem Bloomova filtru je konstantní časová složitost operací vkládání a dotazování. Čas potřebný pro tyto operace by měl záviset výhradně na počtu hashovacích funkcí (k) a nikoliv na celkovém počtu prvků ve filtru (n). Abychom tento předpoklad experimentálně ověřili, byla sledována latence operací `Add` a `Contains` pro kapacity rostoucí od 10^4 do $4 \cdot 10^7$ prvků.

Jak jasně demonstruje Obrázek 6.1, v intervalu od 10^4 do 10^6 prvků si při vkládání algoritmus udržuje naprosto stabilní výkon. Přestože se objem dat stonásobně zvětšil, průměrný čas operace vkládání zůstal konstantní (přibližně 79 ns). Tento fakt exaktně potvrzuje časovou složitost $\mathcal{O}(1)$ v ideálních podmínkách, kdy se alokované bitové pole plně vejde do rychlé vyrovnávací paměti procesoru.



Obrázek 6.1: Závislost času operací na kapacitě filtru

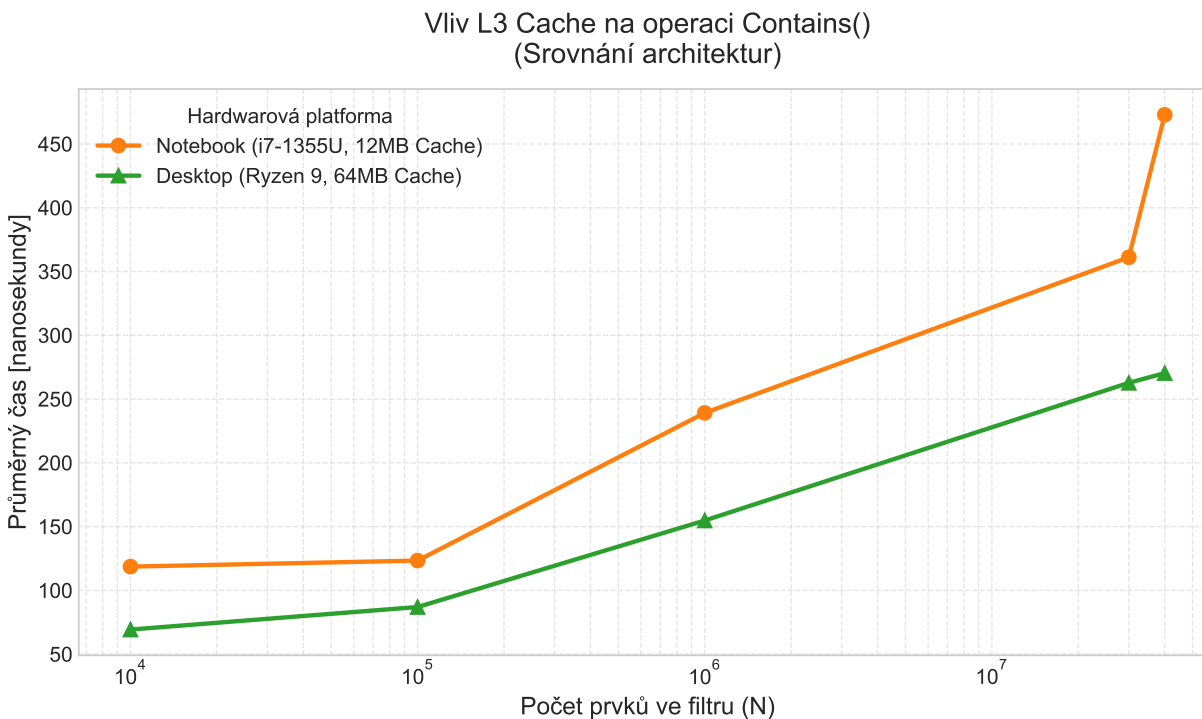
Zlom však nastává u extrémních kapacit ($3 \cdot 10^7$ a více prvků). V tomto bodě je zřejmý prudký nárůst latence. Tento jev není anomálií algoritmu, nýbrž nevyhnutelným důsledkem hardwarových limitů. Filtr konfigurovaný pro 30 milionů prvků s chybovostí 1 % vyžaduje alokaci paměti přesahující 34 MB. Vzhledem k tomu, že notebook disponuje pouze 12 MB L3 Cache, procesor již nedokáže udržet strukturu na čipu. Technika vícenásobného hashování navíc přistupuje k bitům pseudonáhodně, což znemožňuje přednačítání instrukcí do mezipaměti. Procesor je tak nucen načítat data z výrazně pomalejší operační paměti, čímž reálný čas operace vzroste více než trojnásobně.

Ověření vlivu architektury procesoru

K ověření hypotézy, že popsaná degradace výkonu je čistě hardwarového původu, byla identická sada testů spuštěna na desktopové stanici vybavené procesorem se 64 MB L3 Cache. Srovnání obou architektur pro operaci dotazování ilustruje Obrázek 6.2.

Z grafu je patrné, že architektura disponující podstatně větší kapacitou L3 Cache dokáže oddálit použití operační paměti a zpomalit tak operace filtru. Zatímco notebookový procesor vykazuje strmý nárůst latence již u hranice $3 \cdot 10^7$ prvků, procesor AMD Ryzen 9 je díky 64 MB Cache schopen udržet strukturu podstatně déle na čipu a zpracovávat dotazy výrazně rychleji.

Pro lepší porovnání závislosti výkonu pravděpodobnostních struktur na hardwaru počítače byla zvolena operace vkládání (Add). Tato operace je pro profilování paměťové propustnosti vhodnější



Obrázek 6.2: Srovnání vlivu velikosti L3 Cache na čase provedení operace Contains

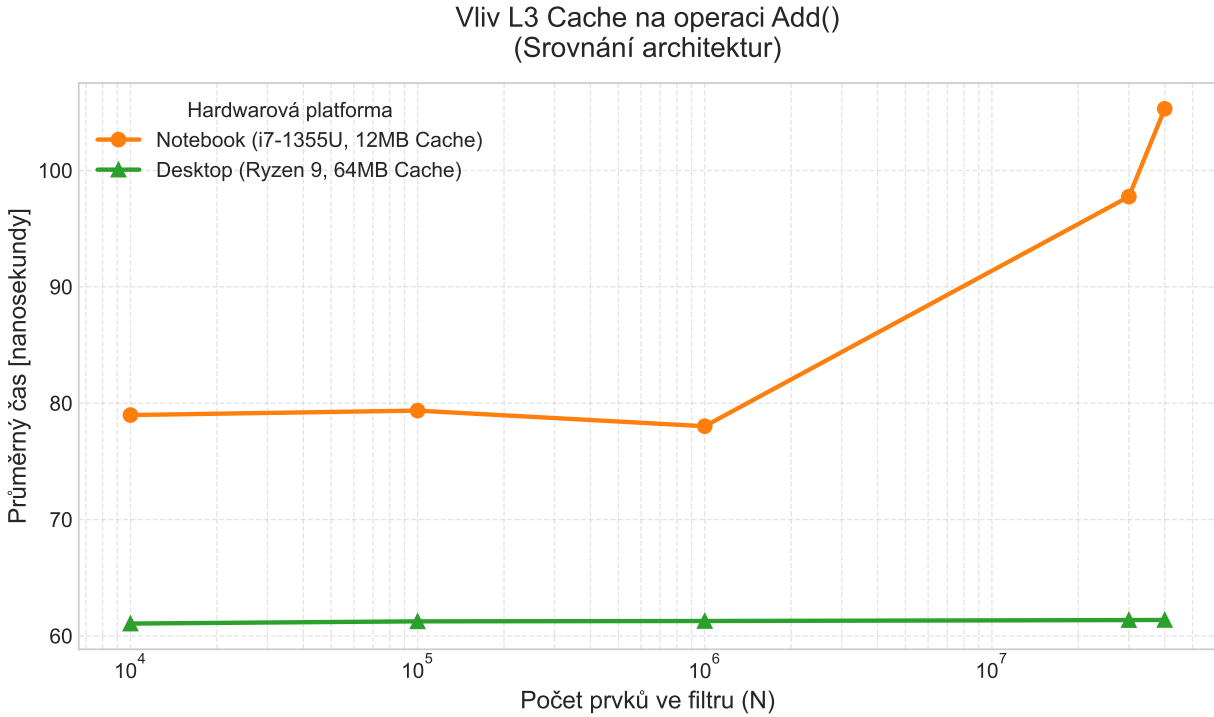
než dotazování (**Contains**), neboť u ní nedopočítáváme náhodné indexy prvků a tudíž je podstatně časově méně náročná.

Následující Obrázek 6.3 ilustruje tento masivní rozdíl ve změnách vyrovnávacích pamětí, na kterých je Bloomův filtr umístěn.

Při analýze grafu je zřetelně vidět fyzikální limit vyrovnávací paměti notebooku. Bloomův filtr pro $4 \cdot 10^7$ prvků s chybovostí 1 % alokuje paměťový blok o velikosti přesahující 45 MB. Zatímco do kapacity 10^6 prvků (cca 1,1 MB) si notebook drží konstantní časovou složitost, při přechodu na extrémní objemy dat je kapacita jeho 12 MB L3 Cache zcela vyčerpána. Procesor je nucen zapisovat data do mnohem pomalejší operační paměti, což se na grafu projevuje zřetelným nárůstem latence ze 79 ns na více než 105 ns. Naproti tomu procesor AMD Ryzen 9 díky své 64 MB vyrovnávací paměti dokáže pojmout celou 45 MB strukturu filtru přímo v cache procesoru. Algoritmus si díky tomu zachovává naprosto dokonalou a neměnnou teoretickou složitost $\mathcal{O}(1)$ i u těch největších měřených sad. Tento experiment jednoznačně prokazuje, že limity Bloomových filtrů v platformě .NET jsou dány i hardwarovou hierarchií pamětí.

6.3.2 Režie vkládání u pokročilých variant filtrů

Při měření rychlostních časů operace vkládání (**Add**) na plné škále kapacit narazilo profilování škálovatelného filtru na zabudovaný problém nástroje BenchmarkDotNet. U měření filtrů, které přesahují



Obrázek 6.3: Srovnání vlivu kapacity L3 Cache na čase provedení operace Add

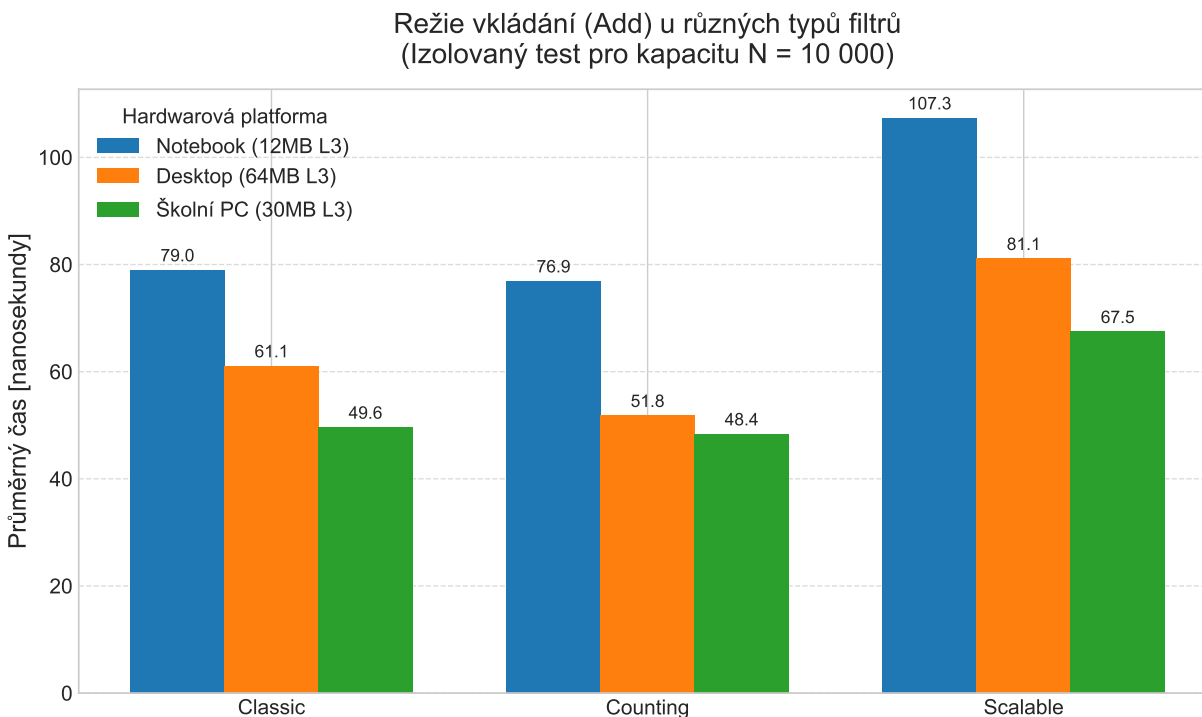
10^5 prvků vyvolá vysoká frekvence vkládání v rámci benchmarkovací smyčky sérii dynamických re-alkací nových vrstev filtru. Tím dochází k velké zátěži Garbage Collectoru, což měřicí framework vyhodnotí jako nestabilitu a výsledky zahodí (ve výsledcích se toto chování vyhodnocuje, jako hodnota NA). Z tohoto důvodu je pro objektivní srovnání režie vkládání vybrán izolovaný vzorek o kapacitě $N = 10\,000$, u kterého nedochází k vícenásobnému přetečení kapacity.

Data zobrazená na Obrázku 6.4 prokazují mimořádnou efektivitu filtru **Counting**. Zápis do pole 8bitových celých čísel (`byte[]`) nevyžaduje v porovnání s bitovými operacemi klasického filtru (**Classic**) prakticky žádný dodatečný procesorový čas a obě struktury dosahují téměř identické rychlosti napříč všemi testovanými architekturami.

U škálovatelného filtru (**Scalable**) je pozorovatelné mírné zpoždění (cca o 25 až 30 %). Toto zpomalení je přímým důsledkem dodatečné režie spojené s kontrolou kapacity aktuální vrstvy a směrováním zápisu do správné vnitřní instance filtru. I přesto se latence vkládání bezpečně drží v řádu desítek nanosekund.

6.3.3 Analýza Worst-Case scénáře u škálovatelného filtru

Při analýze datových struktur určených pro dynamické rozšiřování kapacity hraje klíčovou roli chování systému při tzv. *Worst-Case* scénáři. U škálovatelného (**Scalable**) Bloomova filtru je tímto *Worst-Case* scénářem dotaz na prvek, který se ve filtru nenachází (**Contains (False)**). Následující



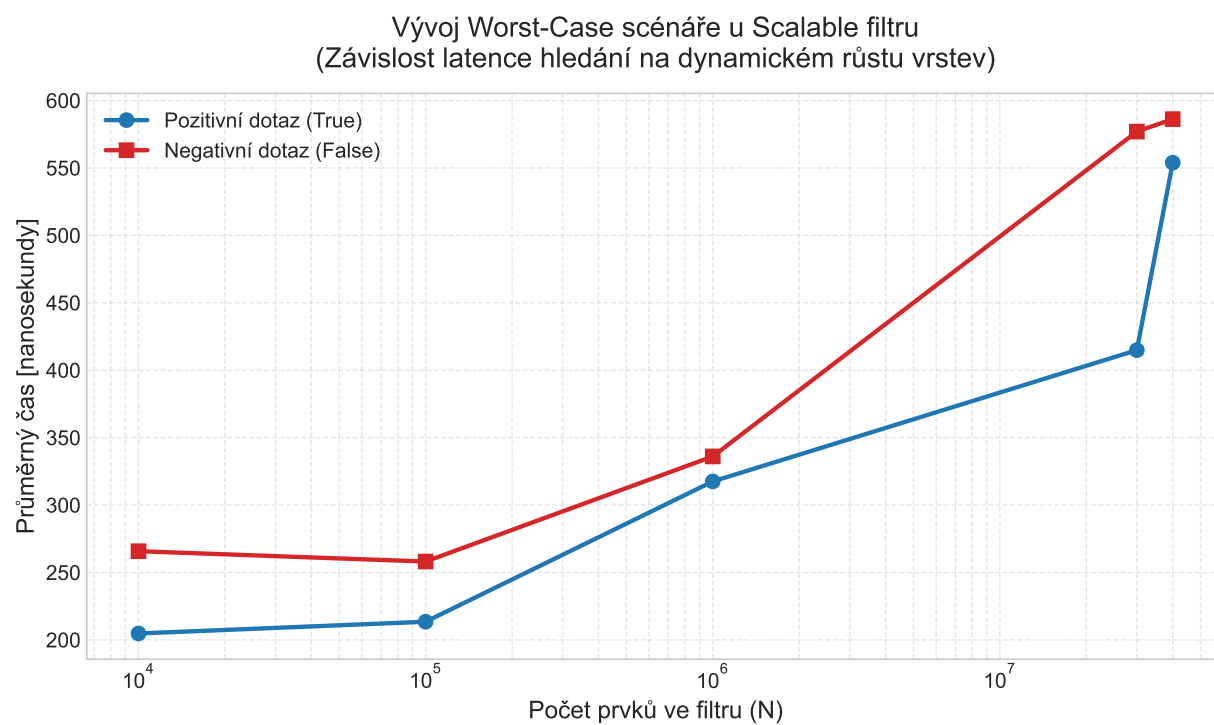
Obrázek 6.4: Porovnání rychlosti vkládání u jednotlivých implementací filtru

graf ilustruje, jak se s rostoucí kapacitou filtru (a tedy s rostoucím počtem jeho vnitřních vrstev) zvětšuje rozdíl rychlosti mezi pozitivním a negativním výsledkem dotazu.

Jak ukazuje Obrázek 6.5, u malých kapacit (např. do 10^5 prvků) jsou časy pro kladnou a zápornou odpověď téměř identické. V této fázi filtr obsahuje pouze jednu vrstvu, a proto se dotazování chová jako standardní klasický filtr.

Se zvyšujícím se objemem vkládaných dat však filtr začíná dynamicky alokovat další vrstvy. Pokud se dotazujeme na prvek, který ve filtru existuje (**Contains = True**), vyhledávací algoritmus iteruje vrstvami, a jakmile v jedné z nich prvek nalezne, provede předčasné ukončení a vrátí kladný výsledek. Průměrná latence u $30 \cdot 10^6$ prvků se díky tomu drží na úrovni 414 ns.

Zcela odlišná je ovšem situace u dotazu na neexistující prvek (**False**). Aby algoritmus mohl definitivně vyloučit přítomnost prvku v celé struktuře, musí projít všechny dynamicky vytvořené vrstvy. Pro každou další vrstvu filtru musí znovu vypočítat hashovací indexy a znovu přistoupit do paměti. Na grafu je jasně patrné, že latence negativního dotazu stoupá s každou novou vrstvou podstatně strměji a u extrémních kapacit dosahuje hodnoty až 576 ns.



Obrázek 6.5: Rozdíl v časech provedení operace Contains pro pozitivní a negativní výsledek u Škálovatelné verze filtru

Kapitola 7

Závěr

Cílem bakalářské práce byla kompletní analýza pravděpodobnostních datových struktur založených na Bloomově filtru a jejich následná softwarová implementace pro platformu .NET.

Výsledkem praktické části je plně funkční a distribuovatelná knihovna napsaná v jazyce C#, jež vývojářům poskytuje jednu klasickou a dvě specializované varianty této struktury. Vedle standardního filtru, optimalizovaného pro minimální paměťovou stopu, knihovna nabízí počítací variantu (CBF) umožňující bezpečné odstranění prvků a škálovatelnou variantu (SBF), která dokáže dynamicky alokovat paměťovou kapacitu při zachování fixní míry chybovosti.

Nejdůležitější částí celé práce se nakonec ukázalo být samotné experimentální měření. Testy prováděné přes `BenchmarkDotNet` sice spolehlivě potvrdily teoretickou časovou složitost $\mathcal{O}(1)$, mnohem přínosnější ale bylo sledovat reálné limity hardwaru na různých procesorech. Z výsledků je totiž patrné, že zpoždění filtru kriticky závisí na kapacitě L3 Cache. Jakmile vložená data nabydou do takových rozměrů (typicky desítky milionů prvků), že se bitové pole do L3 cache zkrátka nevejde, procesor musí začít číst z pomalejší paměti RAM. V ten moment dochází k logickému propadu výkonu.

Celá vytvořená knihovna je aktuálně dostupná v oficiálním repozitáři NuGet a vývojáři ji mohou nasadit do svých projektů. Jedním z kroků, jak knihovnu ještě vylepšit by mohlo být přidání širší palety nekryptografických hashovacích funkcí. Další, poměrně náročnou architektonickou výzvou by pak byl přepis vnitřní synchronizace na zcela neblokující (*lock-free*) model. To by filtru umožnilo zpracovávat data ještě rychleji v silně paralelním prostředí.

Provedené experimenty pak potvrzují jednu zásadní věc: při psaní vysoce optimalizovaného kódu se nelze spoléhat jen na asymptotickou složitost algoritmu z teoretického pohledu. Abychom dosáhli maximálního reálného výkonu, musíme přímo zohlednit to, jak cílový hardware pracuje se svou hierarchií pamětí.

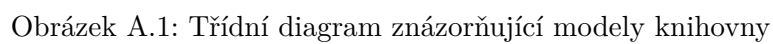
Literatura

1. BLOOM, Burton H. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM* [online]. 1970-07, roč. 13, č. 7, s. 422–426 [cit. 2026-04-01]. ISSN 0001-0782, ISSN 1557-7317. Dostupné z DOI: 10.1145/362686.362692.
2. LI FAN; PEI CAO; ALMEIDA, J.; BRODER, A.Z. Summary cache: a scalable wide-area Web cache sharing protocol. *IEEE/ACM Transactions on Networking* [online]. 2000-06, roč. 8, č. 3, s. 281–293 [cit. 2026-04-01]. ISSN 10636692. Dostupné z DOI: 10.1109/90.851975.
3. ALMEIDA, Paulo Sérgio; BAQUERO, Carlos; PREGUIÇA, Nuno; HUTCHISON, David. Scalable Bloom Filters. *Information Processing Letters* [online]. 2007-03, roč. 101, č. 6, s. 255–261 [cit. 2026-04-01]. ISSN 00200190. Dostupné z DOI: 10.1016/j.ip1.2006.10.007.
4. WULF, Wm. A.; MCKEE, Sally A. Hitting the memory wall: implications of the obvious. *ACM SIGARCH Computer Architecture News* [online]. 1995-03, roč. 23, č. 1, s. 20–24 [cit. 2026-04-01]. ISSN 0163-5964. Dostupné z DOI: 10.1145/216585.216588.
5. BRODER, Andrei; MITZENMACHER, Michael. Network Applications of Bloom Filters: A Survey. *Internet Mathematics* [online]. 2004-01, roč. 1, č. 4, s. 485–509 [cit. 2026-04-05]. ISSN 1542-7951, ISSN 1944-9488. Dostupné z DOI: 10.1080/15427951.2004.10129096.
6. SEDGEWICK, Robert; WAYNE, Kevin Daniel. *Algorithms*. 4th ed. Upper Saddle River: Addison-Wesley, 2011. ISBN 9780321573513.
7. FLAJOLET, Philippe; FUSY, Éric; GANDOUE, Olivier; MEUNIER, Frédéric. HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm. *Discrete Mathematics & Theoretical Computer Science* [online]. 2007-01, roč. DMTCS Proceedings vol. AH,... Č. Proceedings, s. 3545 [cit. 2026-04-01]. ISSN 1365-8050. Dostupné z DOI: 10.46298/dmtcs.3545.
8. CORMODE, Graham; MUTHUKRISHNAN, S. An improved data stream summary: the count-min sketch and its applications. *Journal of Algorithms* [online]. 2005-04, roč. 55, č. 1, s. 58–75 [cit. 2026-04-01]. ISSN 01966774. Dostupné z DOI: 10.1016/j.jalgor.2003.12.001.
9. DANG, Quynh. Changes in Federal Information Processing Standard (FIPS) 180-4, Secure Hash Standard. *Cryptologia* [online]. 2013-01, roč. 37, č. 1, s. 69–73 [cit. 2026-04-05]. ISSN 0161-1194, ISSN 1558-1586. Dostupné z DOI: 10.1080/01611194.2012.687431.

10. APPLEBY, Austin. *MurmurHash* [online]. 2008. [cit. 2024-04-01]. Dostupné z: <https://github.com/aappleby/smhasher>.
11. COLLET, Yann. *xxHash: Extremely fast non-cryptographic hash algorithm* [online]. 2012. [cit. 2024-04-01]. Dostupné z: <https://github.com/Cyan4973/xxHash>.
12. MITZENMACHER, Michael; UPFAL, Eli. *Probability and computing: randomization and probabilistic techniques in algorithms and data analysis*. 2nd ed. Cambridge: Cambridge university press, 2017. ISBN 9781107154889.
13. TARKOMA, Sasu; ROTHENBERG, Christian Esteve; LAGERSPETZ, Eemil. Theory and Practice of Bloom Filters for Distributed Systems. *IEEE Communications Surveys & Tutorials* [online]. 2012, roč. 14, č. 1, s. 131–155 [cit. 2026-04-05]. ISSN 1553-877X. Dostupné z DOI: 10.1109/SURV.2011.031611.00024.
14. MICROSOFT CORPORATION. *C# identifier naming rules and conventions* [online]. 2024. [cit. 2025-11-10]. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/identifier-names>.
15. GAMMA, Erich (ed.). *Design patterns: elements of reusable object-oriented software*. 39. printing. Boston, Mass. Munich: Addison-Wesley, 2011. Addison-Wesley professional computing series. ISBN 9780201633610.
16. KIRSCH, Adam; MITZENMACHER, Michael. Less hashing, same performance: Building a better Bloom filter. *Random Structures & Algorithms* [online]. 2008-09, roč. 33, č. 2, s. 187–218 [cit. 2026-04-05]. ISSN 1042-9832, ISSN 1098-2418. Dostupné z DOI: 10.1002/rsa.20208.

Příloha A

Velké obrázky a tabulky



Metoda a operace	n	$\bar{t}$ [ns]	σ_t [ns]	$A[B]$
Classic: Add	10 000	49,60	0,11	104
Counting: Add	10 000	48,36	0,13	104
Scalable: Add	10 000	67,46	0,16	120
Classic: Contains (True)	10 000	54,91	0,42	91
Counting: Contains (True)	10 000	53,20	0,10	91
Scalable: Contains (True)	10 000	112,60	0,12	189
Scalable: Contains (True)	10 000	112,60	0,12	189
Classic: Contains (False)	10 000	59,73	0,15	93
Counting: Contains (False)	10 000	62,03	1,29	93
Scalable: Contains (False)	10 000	160,66	0,49	268
Classic: Add	1 000 000	51,40	0,10	104
Counting: Add	1 000 000	50,77	0,68	104
Scalable: Add	1 000 000	67,23	0,13	112
Classic: Contains (True)	1 000 000	141,33	5,87	92
Counting: Contains (True)	1 000 000	162,64	1,65	92
Scalable: Contains (True)	1 000 000	200,73	1,14	144
Classic: Contains (False)	1 000 000	141,50	0,91	91
Counting: Contains (False)	1 000 000	153,72	0,60	91
Scalable: Contains (False)	1 000 000	213,37	0,78	168
Classic: Add	30 000 000	53,63	0,17	104
Counting: Add	30 000 000	49,04	0,08	104
Scalable: Add	30 000 000	NA	NA	NA
Classic: Contains (True)	30 000 000	269,45	10,15	91
Counting: Contains (True)	30 000 000	263,47	2,86	91
Scalable: Contains (True)	30 000 000	323,33	6,44	125
Classic: Contains (False)	30 000 000	261,61	7,15	91
Counting: Contains (False)	30 000 000	256,81	0,83	91
Scalable: Contains (False)	30 000 000	306,66	0,84	142
Classic: Add	40 000 000	49,92	0,08	104
Counting: Add	40 000 000	49,34	0,10	104
Scalable: Add	40 000 000	64,40	0,15	104
Classic: Contains (True)	40 000 000	257,00	1,76	91
Counting: Contains (True)	40 000 000	266,51	2,98	91
Scalable: Contains (True)	40 000 000	297,74	2,50	124
Classic: Contains (False)	40 000 000	257,28	0,86	91
Counting: Contains (False)	40 000 000	263,02	1,09	91
Scalable: Contains (False)	40 000 000	313,77	0,61	142

Tabulka A.1: Data výkonnostních testů z Pracovního zařízení 1 (Intel Core i7-13700KF)

Metoda a operace	n	$\bar{t}$ [ns]	σ_t [ns]	$A[B]$
Classic: Add	10 000	61,06	0,20	104
Counting: Add	10 000	51,79	0,17	104
Scalable: Add	10 000	81,09	0,52	120
Classic: Contains (True)	10 000	69,28	1,32	91
Counting: Contains (True)	10 000	64,88	0,24	91
Scalable: Contains (True)	10 000	131,02	0,42	189
Classic: Contains (False)	10 000	70,02	0,32	93
Counting: Contains (False)	10 000	70,29	0,21	93
Scalable: Contains (False)	10 000	180,56	0,67	268
Classic: Add	1 000 000	61,28	0,17	104
Counting: Add	1 000 000	51,58	0,15	104
Scalable: Add	1 000 000	NA	NA	NA
Classic: Contains (True)	1 000 000	154,79	1,59	92
Counting: Contains (True)	1 000 000	163,69	1,08	92
Scalable: Contains (True)	1 000 000	205,54	3,49	144
Classic: Contains (False)	1 000 000	159,54	1,80	91
Counting: Contains (False)	1 000 000	169,02	2,10	91
Scalable: Contains (False)	1 000 000	224,94	1,25	168
Classic: Add	30 000 000	61,36	0,24	104
Counting: Add	30 000 000	51,88	0,13	104
Scalable: Add	30 000 000	NA	NA	NA
Classic: Contains (True)	30 000 000	262,72	1,26	91
Counting: Contains (True)	30 000 000	258,86	0,84	91
Scalable: Contains (True)	30 000 000	322,16	0,64	125
Classic: Contains (False)	30 000 000	262,28	0,82	91
Counting: Contains (False)	30 000 000	259,83	1,31	91
Scalable: Contains (False)	30 000 000	337,44	1,05	142
Classic: Add	40 000 000	61,37	0,15	104
Counting: Add	40 000 000	52,27	0,18	104
Scalable: Add	40 000 000	73,25	0,23	104
Classic: Contains (True)	40 000 000	270,38	1,14	91
Counting: Contains (True)	40 000 000	263,33	0,84	91
Scalable: Contains (True)	40 000 000	328,93	1,66	123
Classic: Contains (False)	40 000 000	267,06	0,76	91
Counting: Contains (False)	40 000 000	262,71	0,65	91
Scalable: Contains (False)	40 000 000	351,28	1,93	142

Tabulka A.2: Data výkonnostních testů z Pracovního zařízení 2 (AMD Ryzen 9 5950X)

Metoda a operace	n	$\bar{t}$ [ns]	σ_t [ns]	A [B]
Classic: Add	10 000	78,98	1,04	104
Counting: Add	10 000	76,91	0,48	104
Scalable: Add	10 000	107,35	0,81	120
Classic: Contains (True)	10 000	118,64	17,81	91
Counting: Contains (True)	10 000	104,74	8,25	91
Scalable: Contains (True)	10 000	204,77	15,41	189
Classic: Contains (False)	10 000	116,71	12,46	93
Counting: Contains (False)	10 000	103,45	1,67	93
Scalable: Contains (False)	10 000	265,80	3,18	268
Classic: Add	1 000 000	78,02	0,55	104
Counting: Add	1 000 000	74,54	0,44	104
Scalable: Add	1 000 000	NA	NA	NA
Classic: Contains (True)	1 000 000	239,15	1,91	92
Counting: Contains (True)	1 000 000	315,81	2,20	91
Scalable: Contains (True)	1 000 000	317,54	4,61	144
Classic: Contains (False)	1 000 000	255,05	4,52	91
Counting: Contains (False)	1 000 000	266,19	2,50	91
Scalable: Contains (False)	1 000 000	336,06	2,52	169
Classic: Add	30 000 000	97,75	11,41	104
Counting: Add	30 000 000	100,62	4,98	104
Scalable: Add	30 000 000	129,95	2,91	104
Classic: Contains (True)	30 000 000	361,19	3,79	91
Counting: Contains (True)	30 000 000	366,77	1,61	91
Scalable: Contains (True)	30 000 000	414,91	2,37	125
Classic: Contains (False)	30 000 000	376,23	8,62	91
Counting: Contains (False)	30 000 000	386,11	1,69	91
Scalable: Contains (False)	30 000 000	576,99	10,11	142
Classic: Add	40 000 000	105,30	2,38	104
Counting: Add	40 000 000	100,86	2,08	104
Scalable: Add	40 000 000	127,69	2,21	104
Classic: Contains (True)	40 000 000	472,90	8,99	91
Counting: Contains (True)	40 000 000	488,18	11,98	91
Scalable: Contains (True)	40 000 000	553,94	18,50	124
Classic: Contains (False)	40 000 000	467,15	3,85	91
Counting: Contains (False)	40 000 000	485,32	8,83	91
Scalable: Contains (False)	40 000 000	586,22	12,29	142

Tabulka A.3: Data výkonnostních testů z Pracovního zařízení 3 (Intel Core i7-1355U)

Příloha B

Elektronické přílohy dostupné skrze odevzdávací systém EDISON

Obsah elektronické přílohy:

- Kompletní zdrojové kódy projektu a vytvořené knihovny v jazyce C#
- Sada automatizovaných jednotkových testů (*Unit Tests*)
- Sada výkonnostních testů (*Benchmarks*)
- Kompletní neformátovaná naměřená data z provedených benchmarkových testů v různých formátech
- Soubor README, který obsahuje popis postupu nastavení a spuštění knihovny